

Name:B.Priyadharshini

Reg No:192311337

SIMATS ENGINEERING — DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LABORATORY MANUAL — MACHINE LEARNING

ITA0610 (Slot A)

Lab 1 — FIND-S Algorithm

Aim: To implement the FIND-S algorithm to find the most specific hypothesis consistent with a given set of training examples stored in a CSV file.

Algorithm: Initialise h to the most specific hypothesis; for each positive example, generalise h minimally on any attribute that h fails to satisfy; ignore negative examples; output final h.

Program&Output

```
import pandas as pd
```

```
data = {  
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],  
    'AirTemp': ['Warm', 'Warm', 'Cold', 'Warm'],  
    'Humidity': ['Normal', 'High', 'High', 'High'],  
    'Wind': ['Strong', 'Strong', 'Strong', 'Weak'],  
    'Water': ['Warm', 'Warm', 'Warm', 'Warm'],  
    'Forecast': ['Same', 'Same', 'Change', 'Change'],  
    'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']  
}
```

```
df = pd.DataFrame(data)
df.to_csv('training_data.csv', index=False)
```

```
data = pd.read_csv('training_data.csv')
```

```
h = ['0'] * (len(data.columns) - 1)
```

```
for i in range(len(data)):
    if data.iloc[i, -1] == 'Yes':
        for j in range(len(h)):
            if h[j] == '0':
                h[j] = data.iloc[i, j]
            elif h[j] != data.iloc[i, j]:
                h[j] = '?'
```

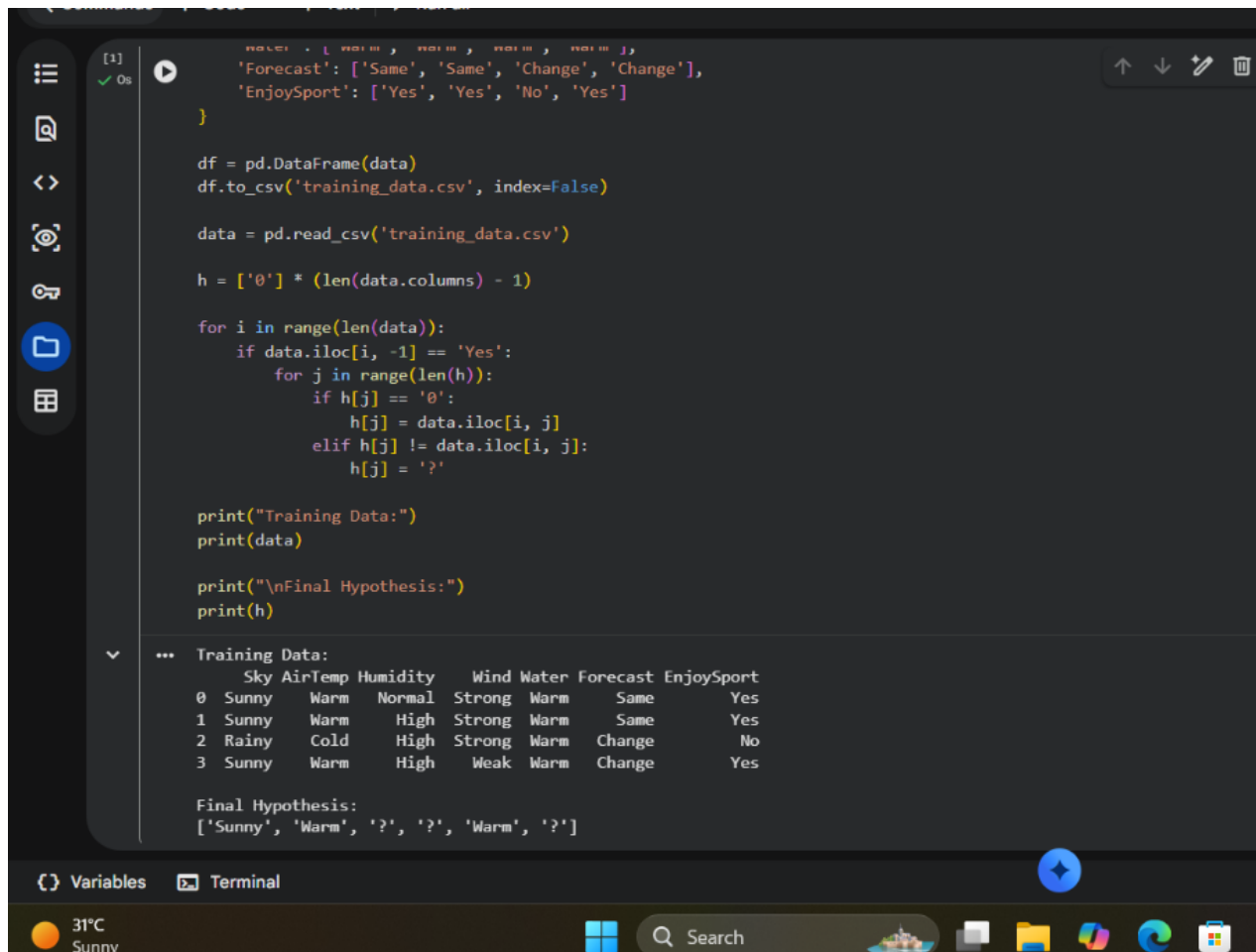
```
print("Training Data:")
```

```
print(data)
```

```
print("\nFinal Hypothesis:")
```

```
print(h)
```

OUTPUT:



```
[1] ✓ 0s
Forecast': ['Same', 'Same', 'Change', 'Change'],
EnjoySport': ['Yes', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)
df.to_csv('training_data.csv', index=False)

data = pd.read_csv('training_data.csv')

h = ['0'] * (len(data.columns) - 1)

for i in range(len(data)):
    if data.iloc[i, -1] == 'Yes':
        for j in range(len(h)):
            if h[j] == '0':
                h[j] = data.iloc[i, j]
            elif h[j] != data.iloc[i, j]:
                h[j] = '?'

print("Training Data:")
print(data)

print("\nFinal Hypothesis:")
print(h)
```

Training Data:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Weak	Warm	Change	Yes

Final Hypothesis:

```
['Sunny', 'Warm', '?', '?', 'Warm', '?']
```

Elimination Algorithm

Aim: To implement the Candidate-Elimination algorithm and output the S and G boundaries of the version space for a given training set.

Algorithm: Initialise S to the most specific and G to the most general hypotheses; for positive examples generalise S and prune G; for negative examples specialise G and prune S; output final S and G.

```
import pandas as pd
```

```
data = {
    'Sky': ['Sunny', 'Sunny', 'Rainy', 'Sunny'],
    'AirTemp': ['Warm', 'Warm', 'Cold', 'Warm'],
```

```
'Humidity': ['Normal', 'High', 'High', 'High'],  
'Wind': ['Strong', 'Strong', 'Strong', 'Weak'],  
'Water': ['Warm', 'Warm', 'Warm', 'Warm'],  
'Forecast': ['Same', 'Same', 'Change', 'Change'],  
'EnjoySport': ['Yes', 'Yes', 'No', 'Yes']  
}
```

```
df = pd.DataFrame(data)  
df.to_csv('training_data.csv', index=False)
```

```
data = pd.read_csv('training_data.csv')
```

```
attributes = list(data.columns[:-1])
```

```
S = ['0'] * len(attributes)
```

```
G = ['?'] * len(attributes)
```

```
for i in range(len(data)):  
    example = list(data.iloc[i, :-1])  
    target = data.iloc[i, -1]
```

```
if target == 'Yes':  
    for j in range(len(S)):  
        if S[j] == '0':  
            S[j] = example[j]  
        elif S[j] != example[j]:  
            S[j] = '?'
```

```
    for j in range(len(G)):
        if G[j] != '?' and G[j] != S[j]:
            G[j] = '?'

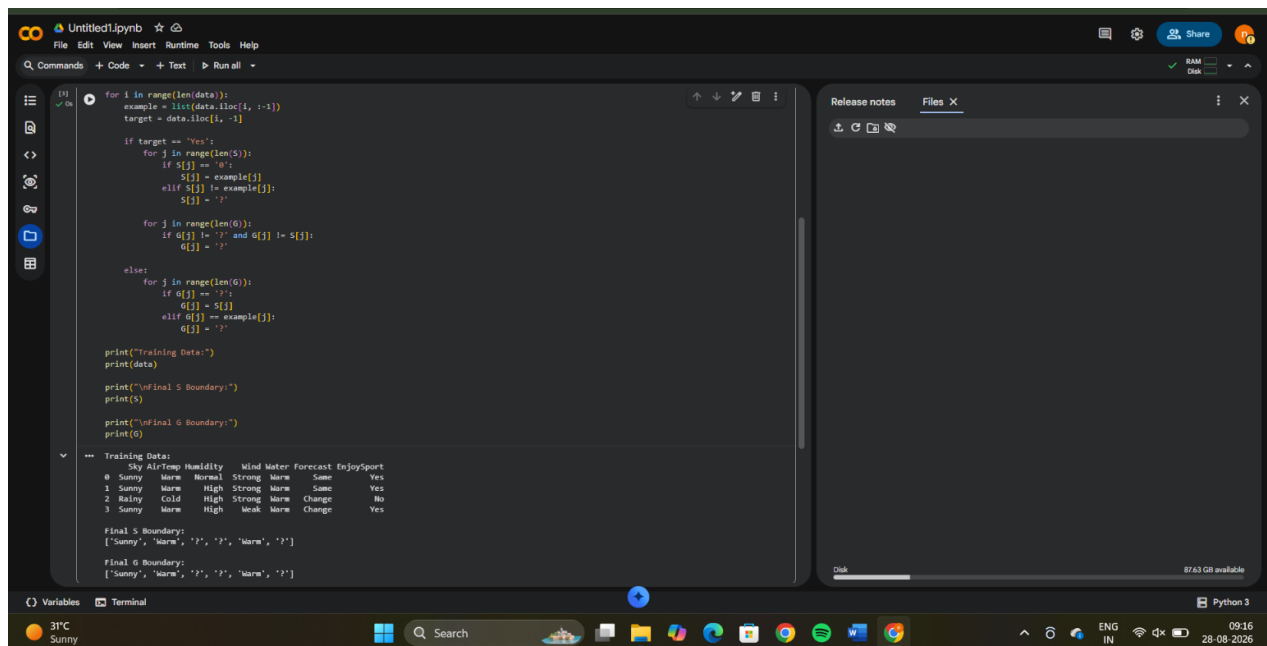
    else:
        for j in range(len(G)):
            if G[j] == '?':
                G[j] = S[j]
            elif G[j] == example[j]:
                G[j] = '?'

print("Training Data:")
print(data)

print("\nFinal S Boundary:")
print(S)

print("\nFinal G Boundary:")
print(G)
```

OUTPUT:



```
for i in range(len(data)):
    example = list(data.iloc[i, :1])
    target = data.iloc[i, -1]

    if target == 'Yes':
        for j in range(len(S)):
            if S[j] == '0':
                S[j] = example[j]
                elif S[j] != example[j]:
                    S[j] = '?'

        for j in range(len(G)):
            if G[j] != '?' and G[j] != S[j]:
                G[j] = '?'

    else:
        for j in range(len(G)):
            if G[j] == '?':
                G[j] = S[j]
                elif G[j] != example[j]:
                    G[j] = '?'

print("Training Data:")
print(data)

print("Final S Boundary:")
print(S)

print("Final G Boundary:")
print(G)
```

Training Data:

	Sky	AirTemp	Humidity	Wind	Water	Forecast	EnjoySport
0	Sunny	Warm	Normal	Strong	Warm	Same	Yes
1	Sunny	Warm	High	Strong	Warm	Same	Yes
2	Rainy	Cold	High	Strong	Warm	Change	No
3	Sunny	Warm	High	Weak	Warm	Change	Yes

Final S Boundary:
['Sunny', 'Warm', '?', '?', 'Warm', '?']

Final G Boundary:
['Sunny', 'Warm', '?', '?', 'Warm', '?']

Lab 3 — Decision Tree Learning (ID3) from Scratch

Aim: To implement the ID3 algorithm to build a decision tree using entropy and information gain, and classify a new sample.

Algorithm: Compute entropy of the dataset; for each attribute compute information gain; select the attribute with maximum gain as the decision node; recurse on subsets until pure or attributes exhausted.

Program:

```
import pandas as pd
```

```
import math
```

```
data = {
```

```
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain', 'Overcast', 'Sunny',  
               'Sunny', 'Rain', 'Sunny', 'Overcast', 'Overcast', 'Rain'],
```

```
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool', 'Cool', 'Mild', 'Cool', 'Mild',  
                   'Mild', 'Mild', 'Hot', 'Mild'],
```

```
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal', 'Normal', 'High',  
                'Normal', 'Normal', 'Normal', 'High', 'Normal', 'High'],
```

```
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong', 'Weak', 'Weak',  
            'Weak', 'Strong', 'Strong', 'Weak', 'Strong'],
```

```
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No', 'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes', 'Yes', 'No']  
}
```

```
df = pd.DataFrame(data)
```

```
def entropy(target):  
    values = target.value_counts()  
    total = len(target)  
    return -sum((count / total) * math.log2(count / total) for count in values)
```

```
def information_gain(data, attribute, target):  
    total_entropy = entropy(data[target])  
    values = data[attribute].unique()  
    weighted_entropy = 0  
  
    for value in values:  
        subset = data[data[attribute] == value]  
        weighted_entropy += (len(subset) / len(data)) * entropy(subset[target])  
  
    return total_entropy - weighted_entropy
```

```
def id3(data, attributes, target):  
    if len(data[target].unique()) == 1:  
        return data[target].iloc[0]  
  
    if len(attributes) == 0:  
        return data[target].mode()[0]
```

```
gains = {attribute: information_gain(data, attribute, target) for attribute in
attributes}
```

```
best_attribute = max(gains, key=gains.get)
```

```
tree = {best_attribute: {}}
```

```
remaining_attributes = [a for a in attributes if a != best_attribute]
```

```
for value in data[best_attribute].unique():
```

```
    subset = data[data[best_attribute] == value]
```

```
    if len(subset) == 0:
```

```
        tree[best_attribute][value] = data[target].mode()[0]
```

```
    else:
```

```
        tree[best_attribute][value] = id3(
```

```
            subset,
```

```
            remaining_attributes,
```

```
            target
```

```
        )
```

```
return tree
```

```
def predict(tree, sample):
```

```
    if not isinstance(tree, dict):
```

```
        return tree
```

```
    attribute = next(iter(tree))
```

```
    value = sample[attribute]
```



```
    if value in tree[attribute]:
        return predict(tree[attribute][value], sample)
    else:
        return "Unknown"

attributes = list(df.columns[:-1])

tree = id3(df, attributes, 'Play')

print("Training Dataset:")
print(df)

print("\nDecision Tree:")
print(tree)

sample = {
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
}

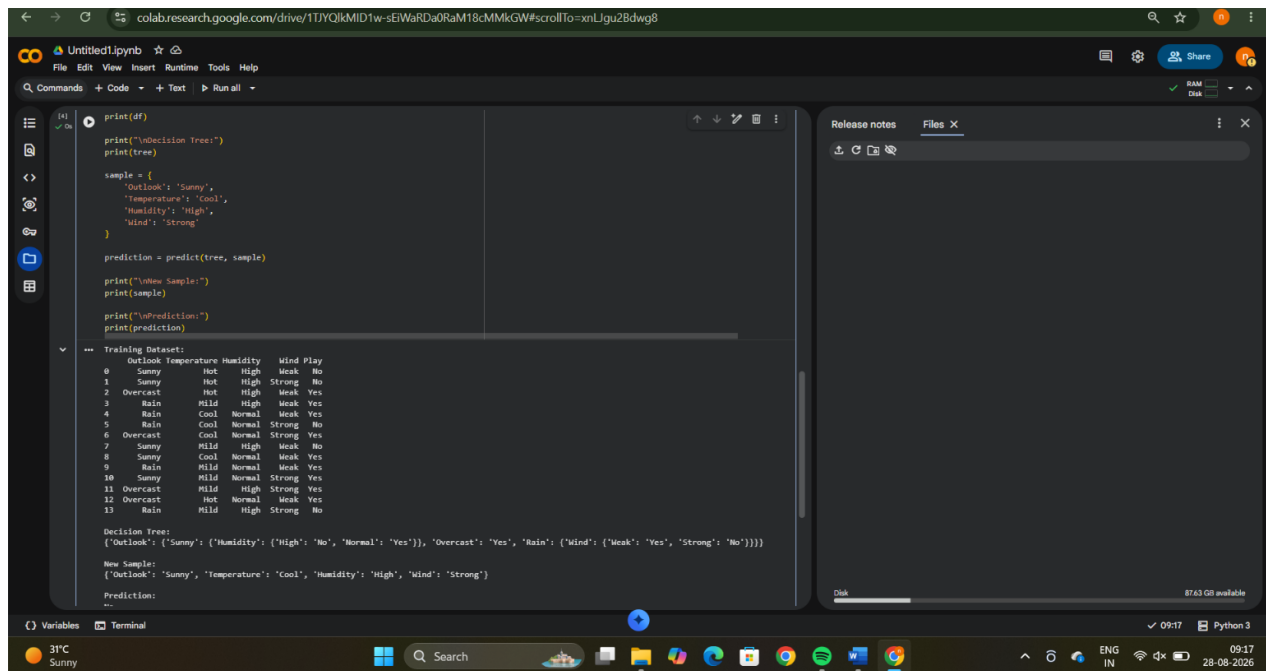
prediction = predict(tree, sample)

print("\nNew Sample:")
print(sample)

print("\nPrediction:")
```

```
print(prediction)
```

OUTPUT:



The screenshot shows a Jupyter Notebook interface with a dark theme. The main code cell contains the following Python code:

```
print(df)
print("\nDecision Tree")
print(tree)

sample = {
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
}

prediction = predict(tree, sample)

print("\nNew Sample:")
print(sample)

print("\nPrediction:")
print(prediction)
```

Below the code, the 'Training Dataset' is displayed as a table:

	Outlook	Temperature	Humidity	Wind	Play
0	Sunny	Hot	High	Weak	No
1	Sunny	Hot	High	Strong	No
2	Overcast	Hot	High	Weak	Yes
3	Rain	Mild	High	Weak	Yes
4	Rain	Cool	Normal	Weak	Yes
5	Rain	Cool	Normal	Strong	No
6	Overcast	Cool	Normal	Strong	Yes
7	Sunny	Mild	High	Weak	No
8	Sunny	Cool	Normal	Weak	Yes
9	Rain	Mild	Normal	Weak	Yes
10	Sunny	Mild	Normal	Strong	Yes
11	Overcast	Mild	High	Strong	Yes
12	Overcast	Hot	Normal	Weak	Yes
13	Rain	Mild	High	Strong	No

Below the table, the 'Decision Tree' structure is shown:

```
Decision Tree:
(['Outlook': ('Sunny': ('Humidity': ('High': ('No', 'Normal': 'Yes')), 'Overcast': 'Yes', 'Rain': ('Wind': ('Weak': 'Yes', 'Strong': 'No')))])

New Sample:
{'Outlook': 'Sunny', 'Temperature': 'Cool', 'Humidity': 'High', 'Wind': 'Strong'}

Prediction:
No
```

The interface also shows a sidebar with 'Release notes' and 'Files' tabs, and a bottom status bar with system information like '31°C Sunny' and '09:17 28-08-2026'.

Lab 4 — Decision Tree using scikit-learn with Visualisation

Aim: To build and visualise a decision tree classifier using scikit-learn on the Iris dataset.

Algorithm: Load dataset; split into train/test; fit DecisionTreeClassifier; evaluate accuracy; plot the tree.

Program:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(criterion='entropy', random_state=42)
```

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nPredicted Values:")
print(y_pred)

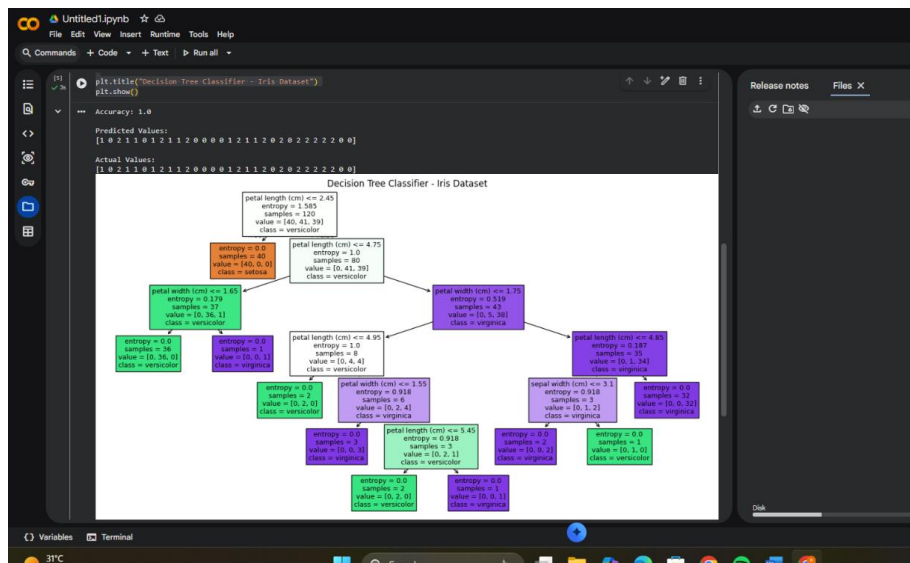
print("\nActual Values:")
print(y_test)

plt.figure(figsize=(15, 8))

plot_tree(
    model,
    feature_names=iris.feature_names,
    class_names=iris.target_names,
    filled=True
)

plt.title("Decision Tree Classifier - Iris Dataset")
plt.show()
```

OUTPUT:



Lab 5 — Single-Layer Perceptron (AND / OR Gate)

Aim: To implement a single-layer perceptron from scratch to learn the AND and OR logic gates.

Algorithm: Initialise weights; for each epoch and training example, compute output using the threshold rule and update weights via $\Delta w_i = \eta(t-o)x_i$ until convergence.

Program:

```
import numpy as np

def perceptron(X, y, learning_rate=0.1, epochs=20):
    weights = np.zeros(X.shape[1])
    bias = 0

    for epoch in range(epochs):
        for i in range(len(X)):
            net = np.dot(X[i], weights) + bias
            output = 1 if net >= 0 else 0
            error = y[i] - output

            weights = weights + learning_rate * error * X[i]
            bias = bias + learning_rate * error

    return weights, bias

def predict(X, weights, bias):
    outputs = []

    for x in X:
        net = np.dot(x, weights) + bias
```

```
output = 1 if net >= 0 else 0
```

```
outputs.append(output)
```

```
return outputs
```

```
X = np.array([
```

```
    [0, 0],
```

```
    [0, 1],
```

```
    [1, 0],
```

```
    [1, 1]
```

```
])
```

```
and_y = np.array([0, 0, 0, 1])
```

```
or_y = np.array([0, 1, 1, 1])
```

```
and_weights, and_bias = perceptron(X, and_y)
```

```
or_weights, or_bias = perceptron(X, or_y)
```

```
and_output = predict(X, and_weights, and_bias)
```

```
or_output = predict(X, or_weights, or_bias)
```

```
print("AND Gate")
```

```
print("Weights:", and_weights)
```

```
print("Bias:", and_bias)
```

```
print("Output:", and_output)
```

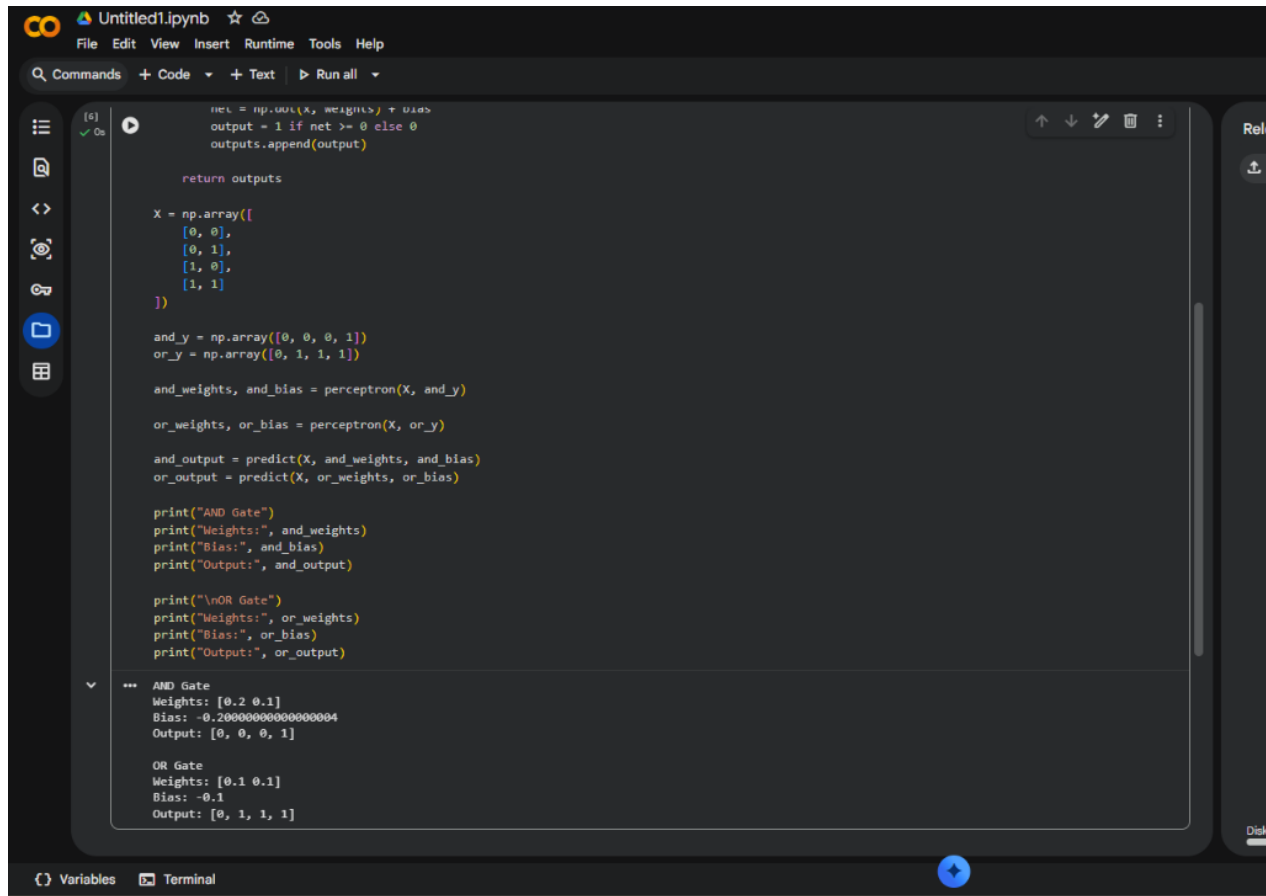
```
print("\nOR Gate")
```

```
print("Weights:", or_weights)
```

```
print("Bias:", or_bias)
```

```
print("Output:", or_output)
```

OUTPUT:



```
net = np.dot(x, weights) + bias
output = 1 if net >= 0 else 0
outputs.append(output)

return outputs

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

and_y = np.array([0, 0, 0, 1])
or_y = np.array([0, 1, 1, 1])

and_weights, and_bias = perceptron(X, and_y)

or_weights, or_bias = perceptron(X, or_y)

and_output = predict(X, and_weights, and_bias)
or_output = predict(X, or_weights, or_bias)

print("AND Gate")
print("Weights:", and_weights)
print("Bias:", and_bias)
print("Output:", and_output)

print("\nOR Gate")
print("Weights:", or_weights)
print("Bias:", or_bias)
print("Output:", or_output)
```

AND Gate
Weights: [0.2 0.1]
Bias: -0.20000000000000004
Output: [0, 0, 0, 1]

OR Gate
Weights: [0.1 0.1]
Bias: -0.1
Output: [0, 1, 1, 1]

Lab 6 — Perceptron using scikit-learn

Aim: To implement a perceptron classifier using scikit-learn's Perceptron on a linearly separable binary dataset.

Algorithm: Load/generate data; fit `sklearn.linear_model.Perceptron`; evaluate accuracy.

Program:

```
import numpy as np
```

```
from sklearn.linear_model import Perceptron
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.metrics import accuracy_score
```

```
X = np.array([  
    [1, 1],  
    [2, 1],  
    [2, 2],  
    [3, 2],  
    [6, 5],  
    [7, 6],  
    [8, 6],  
    [9, 7]  
])
```

```
y = np.array([0, 0, 0, 0, 1, 1, 1, 1])
```

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.25, random_state=42  
)
```

```
model = Perceptron(max_iter=1000, eta0=0.1, random_state=42)
```

```
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Training Data:")
```

```
print(X_train)
```

```
print("\nTesting Data:")
```

```
print(X_test)
```

```
print("\nActual Values:")
```

```
print(y_test)
```

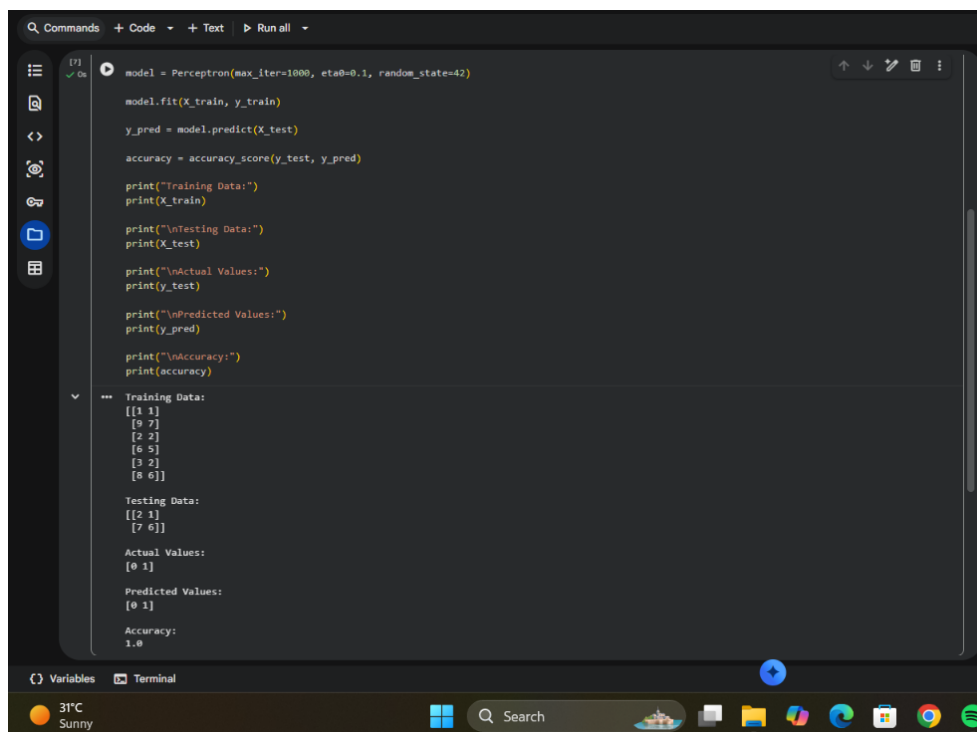
```
print("\nPredicted Values:")
```

```
print(y_pred)
```

```
print("\nAccuracy:")
```

```
print(accuracy)
```

OUTPUT:



```
[7] Out: model = Perceptron(max_iter=1000, eta=0.1, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Training Data:")
print(X_train)

print("\nTesting Data:")
print(X_test)

print("\nActual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:")
print(accuracy)
```

Training Data:

```
[[1 1]
 [9 7]
 [2 2]
 [6 5]
 [3 2]
 [8 6]]
```

Testing Data:

```
[[2 1]
 [7 6]]
```

Actual Values:

```
[0 1]
```

Predicted Values:

```
[0 1]
```

Accuracy:

```
1.0
```


Lab 7 — Multilayer Perceptron with Backpropagation (XOR) from Scratch

Aim: To implement backpropagation from scratch to train a multilayer perceptron to learn the XOR function.

Algorithm: Forward-propagate inputs through hidden and output sigmoid units; compute output error δ ; backpropagate δ to hidden units; update weights using gradient descent with momentum.

Program:

```
import numpy as np
```

```
def sigmoid(x):
```

```
    return 1 / (1 + np.exp(-x))
```

```
def sigmoid_derivative(x):
```

```
    return x * (1 - x)
```

```
X = np.array([
```

```
    [0, 0],
```

```
    [0, 1],
```

```
    [1, 0],
```

```
    [1, 1]
```

```
])
```

```
y = np.array([
```

```
    [0],
```

```
    [1],
```

```
    [1],
```

```
    [0]
```

```
])
```

```
np.random.seed(42)
```

```
input_size = 2
```

```
hidden_size = 4
```

```
output_size = 1
```

```
learning_rate = 0.5
```

```
momentum = 0.9
```

```
epochs = 10000
```

```
W1 = np.random.uniform(-1, 1, (input_size, hidden_size))
```

```
b1 = np.zeros((1, hidden_size))
```

```
W2 = np.random.uniform(-1, 1, (hidden_size, output_size))
```

```
b2 = np.zeros((1, output_size))
```

```
dW1_prev = np.zeros_like(W1)
```

```
db1_prev = np.zeros_like(b1)
```

```
dW2_prev = np.zeros_like(W2)
```

```
db2_prev = np.zeros_like(b2)
```

```
for epoch in range(epochs):
```

```
    hidden_input = np.dot(X, W1) + b1
```

```
    hidden_output = sigmoid(hidden_input)
```

output_input = np.dot(hidden_output, W2) + b2

output = sigmoid(output_input)

output_error = y - output

output_delta = output_error * sigmoid_derivative(output)

hidden_error = np.dot(output_delta, W2.T)

hidden_delta = hidden_error * sigmoid_derivative(hidden_output)

**dW2 = learning_rate * np.dot(hidden_output.T, output_delta) + momentum *
dW2_prev**

**db2 = learning_rate * np.sum(output_delta, axis=0, keepdims=True) + momentum *
db2_prev**

dW1 = learning_rate * np.dot(X.T, hidden_delta) + momentum * dW1_prev

**db1 = learning_rate * np.sum(hidden_delta, axis=0, keepdims=True) + momentum *
db1_prev**

W2 += dW2

b2 += db2

W1 += dW1

b1 += db1

dW2_prev = dW2

db2_prev = db2

dW1_prev = dW1

db1_prev = db1

```
hidden_output = sigmoid(np.dot(X, W1) + b1)
```

```
final_output = sigmoid(np.dot(hidden_output, W2) + b2)
```

```
predictions = (final_output >= 0.5).astype(int)
```

```
print("XOR Input:")
```

```
print(X)
```

```
print("\nActual Output:")
```

```
print(y.flatten())
```

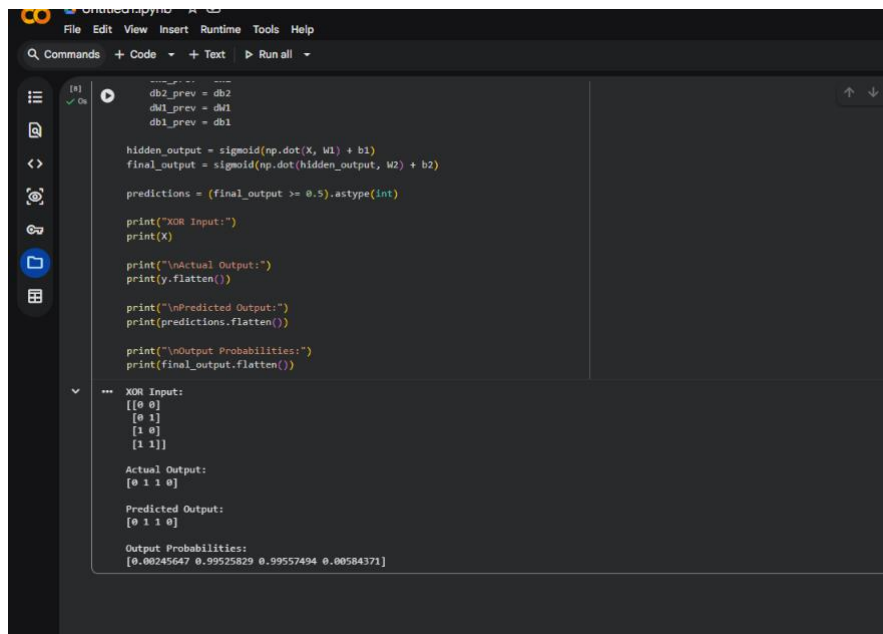
```
print("\nPredicted Output:")
```

```
print(predictions.flatten())
```

```
print("\nOutput Probabilities:")
```

```
print(final_output.flatten())
```

OUTPUT:



The screenshot shows a Jupyter Notebook interface with a dark theme. The code cell contains the following Python code:

```
db2_prev = db2
db1_prev = db1
db1_prev = db1

hidden_output = sigmoid(np.dot(X, W1) + b1)
final_output = sigmoid(np.dot(hidden_output, W2) + b2)

predictions = (final_output >= 0.5).astype(int)

print("XOR Input:")
print(X)

print("\nActual Output:")
print(y.flatten())

print("\nPredicted Output:")
print(predictions.flatten())

print("\nOutput Probabilities:")
print(final_output.flatten())
```

The output cell displays the results of the execution:

```
XOR Input:
[[0 0]
 [0 1]
 [1 0]
 [1 1]]

Actual Output:
[0 1 1 0]

Predicted Output:
[0 1 1 0]

Output Probabilities:
[0.00245647 0.99525829 0.99557494 0.00584371]
```

Lab 8 — Backpropagation Neural Network using scikit-learn (MLPClassifier)

Aim: To train a multilayer perceptron classifier using scikit-learn's MLPClassifier on the Iris dataset.

Algorithm: Load data; scale features; fit MLPClassifier with one hidden layer; evaluate on test set.

program:

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

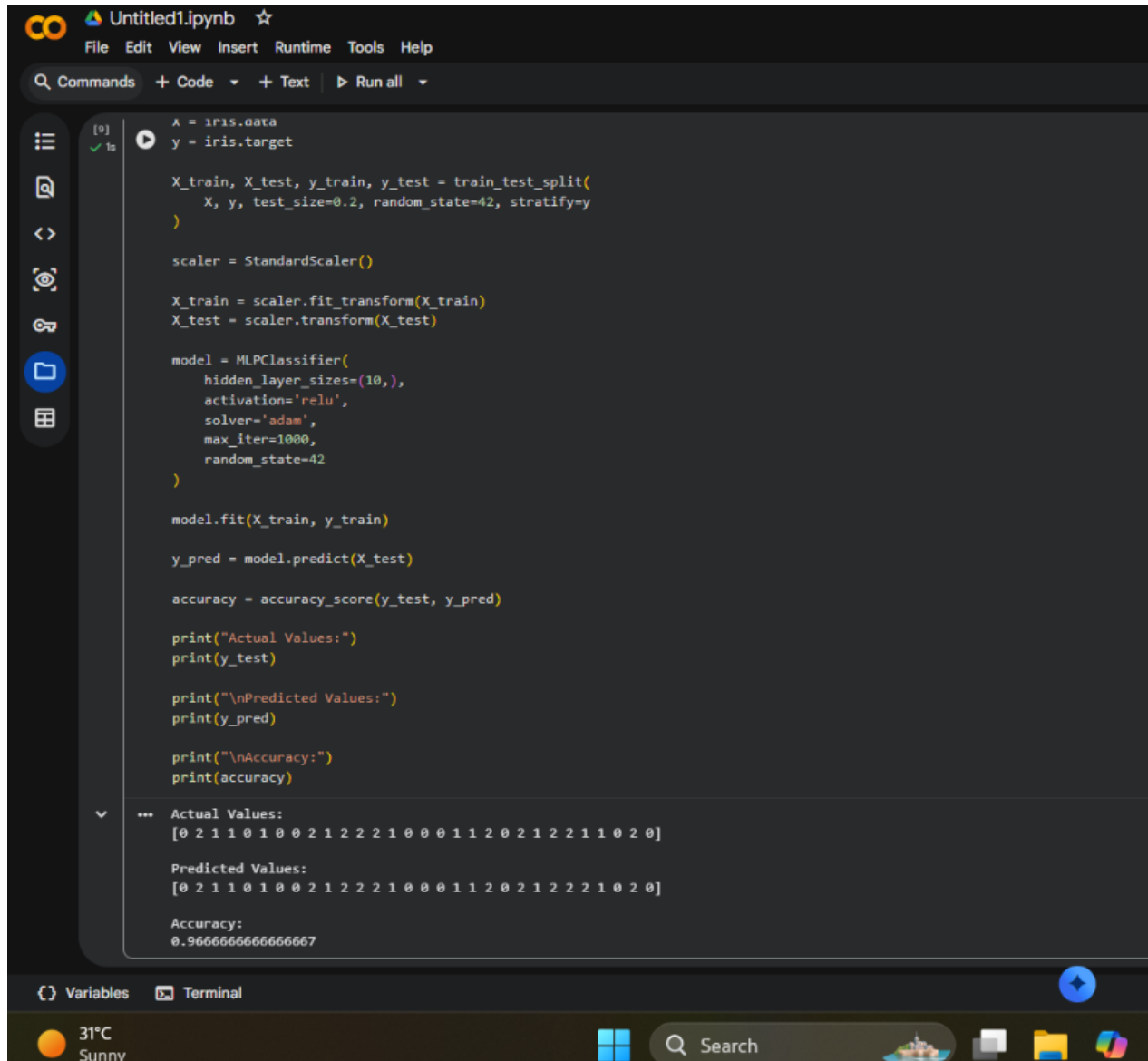
accuracy = accuracy_score(y_test, y_pred)

print("Actual Values:")
print(y_test)
```

```
print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:")
print(accuracy)
```

OUTPUT:



```
Untitled1.ipynb
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[0] ✓ ts
x = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = MLPClassifier(
    hidden_layer_sizes=(10,),
    activation='relu',
    solver='adam',
    max_iter=1000,
    random_state=42
)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)

print("Actual Values:")
print(y_test)

print("\nPredicted Values:")
print(y_pred)

print("\nAccuracy:")
print(accuracy)

--- Actual Values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 1 1 0 2 0]

Predicted Values:
[0 2 1 1 0 1 0 0 2 1 2 2 2 1 0 0 0 1 1 2 0 2 1 2 2 2 1 0 2 0]

Accuracy:
0.9666666666666667

{} Variables Terminal
```

31°C Sunny

Search

Lab 9 — Introduction to Deep Neural Networks using Keras

Aim: To build and train a simple deep neural network (multiple hidden layers) using TensorFlow/Keras on the MNIST digit dataset.

Algorithm: Load and normalise MNIST images; define a sequential model with multiple Dense hidden layers and ReLU activation; compile with Adam optimiser; train and evaluate.

Program:

```
import tensorflow as tf

from tensorflow.keras import Sequential

from tensorflow.keras.layers import Dense, Flatten

from tensorflow.keras.datasets import mnist
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
```

```
x_train = x_train / 255.0
```

```
x_test = x_test / 255.0
```

```
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(10, activation='softmax')
])
```

```
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
```

)

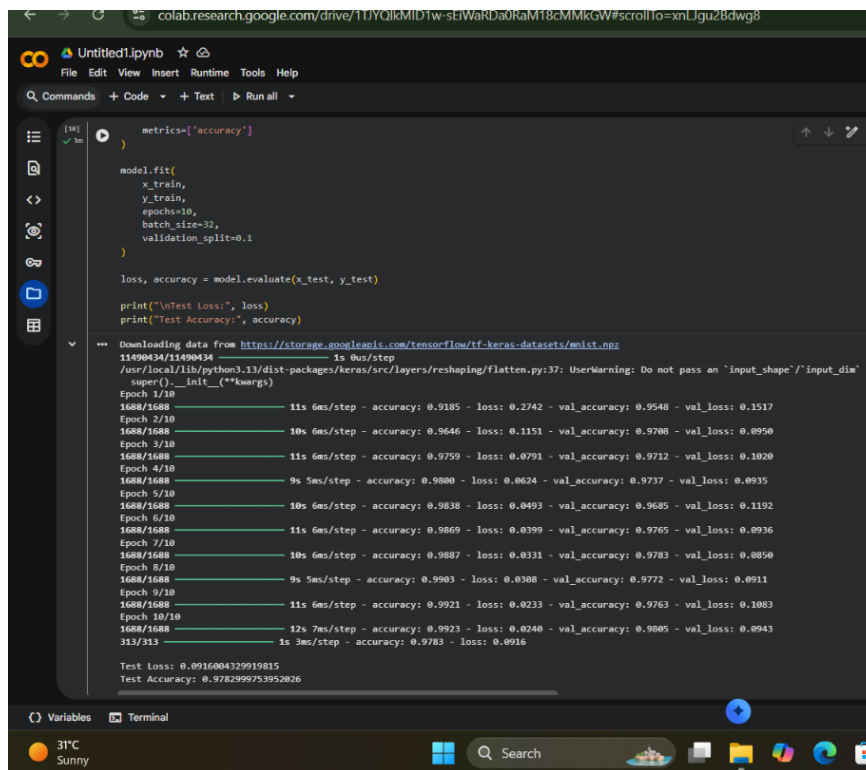
```
model.fit(  
    x_train,  
    y_train,  
    epochs=10,  
    batch_size=32,  
    validation_split=0.1  
)
```

```
loss, accuracy = model.evaluate(x_test, y_test)
```

```
print("\nTest Loss:", loss)
```

```
print("Test Accuracy:", accuracy)
```

OUTPUT:



```
colab.research.google.com/drive/1UJYQIKMID1w-shWaRDa0KaM18cMMkGW#scrollto=xnUgu2Bdwg8  
Untitled1.ipynb  
File Edit View Insert Runtime Tools Help  
Commands + Code + Text Run all  
[10] metrics=['accuracy']  
model.fit(  
    x_train,  
    y_train,  
    epochs=10,  
    batch_size=32,  
    validation_split=0.1  
)  
loss, accuracy = model.evaluate(x_test, y_test)  
print("\nTest Loss:", loss)  
print("Test Accuracy:", accuracy)  
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz  
11490434/11490434 1s 0us/step  
/usr/local/lib/python3.13/dist-packages/keras/src/layers/resizing/flatten.py:37: UserWarning: Do not pass an 'input_shape'/'input_dim' arg  
super().__init__(**kwargs)  
Epoch 1/10 11s 6ms/step - accuracy: 0.0185 - loss: 0.2742 - val_accuracy: 0.9548 - val_loss: 0.1517  
Epoch 2/10 1688/1688 10s 6ms/step - accuracy: 0.9646 - loss: 0.1151 - val_accuracy: 0.9708 - val_loss: 0.0950  
Epoch 3/10 1688/1688 11s 6ms/step - accuracy: 0.9759 - loss: 0.0791 - val_accuracy: 0.9712 - val_loss: 0.1020  
Epoch 4/10 1688/1688 9s 5ms/step - accuracy: 0.9800 - loss: 0.0624 - val_accuracy: 0.9737 - val_loss: 0.0935  
Epoch 5/10 1688/1688 10s 6ms/step - accuracy: 0.9838 - loss: 0.0493 - val_accuracy: 0.9685 - val_loss: 0.1192  
Epoch 6/10 1688/1688 11s 6ms/step - accuracy: 0.9869 - loss: 0.0399 - val_accuracy: 0.9765 - val_loss: 0.0936  
Epoch 7/10 1688/1688 10s 6ms/step - accuracy: 0.9887 - loss: 0.0311 - val_accuracy: 0.9783 - val_loss: 0.0850  
Epoch 8/10 1688/1688 9s 5ms/step - accuracy: 0.9903 - loss: 0.0308 - val_accuracy: 0.9772 - val_loss: 0.0911  
Epoch 9/10 1688/1688 11s 6ms/step - accuracy: 0.9921 - loss: 0.0233 - val_accuracy: 0.9763 - val_loss: 0.1083  
Epoch 10/10 1688/1688 12s 7ms/step - accuracy: 0.9923 - loss: 0.0240 - val_accuracy: 0.9805 - val_loss: 0.0943  
313/313 1s 3ms/step - accuracy: 0.9783 - loss: 0.0916  
Test Loss: 0.0916004329919815  
Test Accuracy: 0.9782999753952026  
31°C Sunny
```


Lab 10 — Genetic Algorithm to Maximise a Function

Aim: To implement a simple genetic algorithm to maximise the function $f(x) = x^2$ over integers 0–31 using bit-string encoding.

Algorithm: Encode candidate x as a 5-bit chromosome; initialise a random population; evaluate fitness $f(x)$; apply roulette-wheel selection, single-point crossover, and mutation across generations.

Program:

```
import random
```

```
def fitness(chromosome):
```

```
    return int(chromosome, 2) ** 2
```

```
def selection(population):
```

```
    fitness_values = [fitness(c) for c in population]
```

```
    total = sum(fitness_values)
```

```
    probabilities = [f / total for f in fitness_values]
```

```
    return random.choices(population, weights=probabilities, k=1)[0]
```

```
def crossover(parent1, parent2):
```

```
    point = random.randint(1, 4)
```

```
    child1 = parent1[:point] + parent2[point:]
```

```
    child2 = parent2[:point] + parent1[point:]
```

```
    return child1, child2
```

```
def mutation(chromosome, mutation_rate=0.01):
```

```
    chromosome = list(chromosome)
```

```
    for i in range(len(chromosome)):
```

```
if random.random() < mutation_rate:  
    chromosome[i] = '1' if chromosome[i] == '0' else '0'
```

```
return ''.join(chromosome)
```

```
population_size = 10
```

```
generations = 20
```

```
mutation_rate = 0.01
```

```
population = [  
    format(random.randint(0, 31), '05b')  
    for _ in range(population_size)  
]
```

```
for generation in range(generations):
```

```
    new_population = []
```

```
    while len(new_population) < population_size:
```

```
        parent1 = selection(population)
```

```
        parent2 = selection(population)
```

```
        child1, child2 = crossover(parent1, parent2)
```

```
        child1 = mutation(child1, mutation_rate)
```

```
        child2 = mutation(child2, mutation_rate)
```

```
    new_population.extend([child1, child2])
```

```
population = new_population[:population_size]
```

```
best = max(population, key=fitness)
```

```
print(  
    "Generation:", generation + 1,  
    "Best x:", int(best, 2),  
    "Fitness:", fitness(best)  
)
```

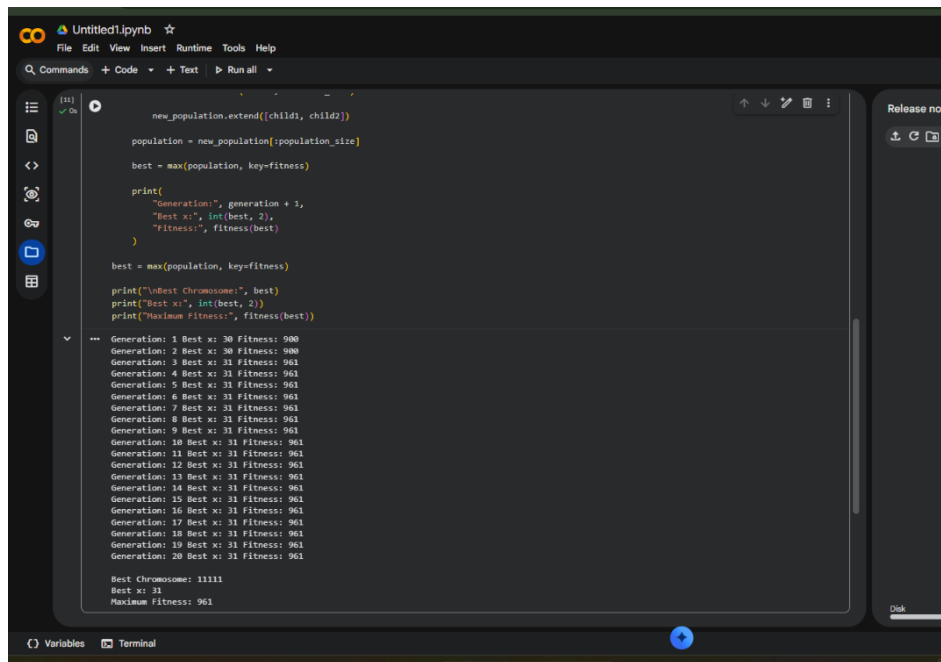
```
best = max(population, key=fitness)
```

```
print("\nBest Chromosome:", best)
```

```
print("Best x:", int(best, 2))
```

```
print("Maximum Fitness:", fitness(best))
```

OUTPUT:



The screenshot shows a Jupyter Notebook interface with a dark theme. The code cell contains a genetic algorithm implementation. The output cell displays the results of 20 generations, showing a steady increase in fitness from 900 to 961. The best chromosome is identified as 11111.

```
new_population.extend([child1, child2])  
population = new_population[:population_size]  
best = max(population, key=fitness)  
  
print(  
    "Generation:", generation + 1,  
    "Best x:", int(best, 2),  
    "Fitness:", fitness(best)  
)  
  
best = max(population, key=fitness)  
  
print("\nBest Chromosome:", best)  
print("Best x:", int(best, 2))  
print("Maximum Fitness:", fitness(best))
```

```
Generation: 1 Best x: 30 Fitness: 900  
Generation: 2 Best x: 30 Fitness: 900  
Generation: 3 Best x: 28 Fitness: 961  
Generation: 4 Best x: 31 Fitness: 961  
Generation: 5 Best x: 31 Fitness: 961  
Generation: 6 Best x: 31 Fitness: 961  
Generation: 7 Best x: 31 Fitness: 961  
Generation: 8 Best x: 31 Fitness: 961  
Generation: 9 Best x: 31 Fitness: 961  
Generation: 10 Best x: 31 Fitness: 961  
Generation: 11 Best x: 31 Fitness: 961  
Generation: 12 Best x: 31 Fitness: 961  
Generation: 13 Best x: 31 Fitness: 961  
Generation: 14 Best x: 31 Fitness: 961  
Generation: 15 Best x: 31 Fitness: 961  
Generation: 16 Best x: 31 Fitness: 961  
Generation: 17 Best x: 31 Fitness: 961  
Generation: 18 Best x: 31 Fitness: 961  
Generation: 19 Best x: 31 Fitness: 961  
Generation: 20 Best x: 31 Fitness: 961  
  
Best Chromosome: 11111  
Best x: 31  
Maximum Fitness: 961
```

Lab 11 — Genetic Algorithm for the 0/1 Knapsack Problem

Aim: To solve the 0/1 knapsack problem using a genetic algorithm to maximise value subject to a weight constraint.

Algorithm: Encode a solution as a binary chromosome (item included/excluded); fitness = total value if weight $\leq$ capacity, else 0 (or penalised); apply selection, crossover, mutation over generations.

Program:

```
import random
```

```
weights = [2, 3, 4, 5, 9]
```

```
values = [3, 4, 5, 8, 10]
```

```
capacity = 10
```

```
population_size = 10
```

```
generations = 20
```

```
mutation_rate = 0.05
```

```
def fitness(chromosome):
```

```
    total_weight = sum(
        chromosome[i] * weights[i] for i in range(len(weights))
    )
```

```
    total_value = sum(
        chromosome[i] * values[i] for i in range(len(values))
    )
```

```
    if total_weight <= capacity:
```

```
        return total_value
```

else:

return 0

def selection(population):

fitness_values = [fitness(c) for c in population]

if sum(fitness_values) == 0:

return random.choice(population)

return random.choices(

population,

weights=fitness_values,

k=1

])[0]

def crossover(parent1, parent2):

point = random.randint(1, len(weights) - 1)

child1 = parent1[:point] + parent2[point:]

child2 = parent2[:point] + parent1[point:]

return child1, child2

def mutation(chromosome):

chromosome = chromosome.copy()

for i in range(len(chromosome)):

```
if random.random() < mutation_rate:  
    chromosome[i] = 1 - chromosome[i]
```

```
return chromosome
```

```
population = [  
    [random.randint(0, 1) for _ in range(len(weights))]  
    for _ in range(population_size)  
]
```

```
for generation in range(generations):
```

```
    new_population = []
```

```
    while len(new_population) < population_size:
```

```
        parent1 = selection(population)
```

```
        parent2 = selection(population)
```

```
        child1, child2 = crossover(parent1, parent2)
```

```
        child1 = mutation(child1)
```

```
        child2 = mutation(child2)
```

```
        new_population.extend([child1, child2])
```

```
    population = new_population[:population_size]
```

```
best = max(population, key=fitness)
```

```
total_weight = sum(  
    best[i] * weights[i] for i in range(len(weights))  
)
```

```
print(  
    "Generation:", generation + 1,  
    "Best Solution:", best,  
    "Weight:", total_weight,  
    "Value:", fitness(best)  
)
```

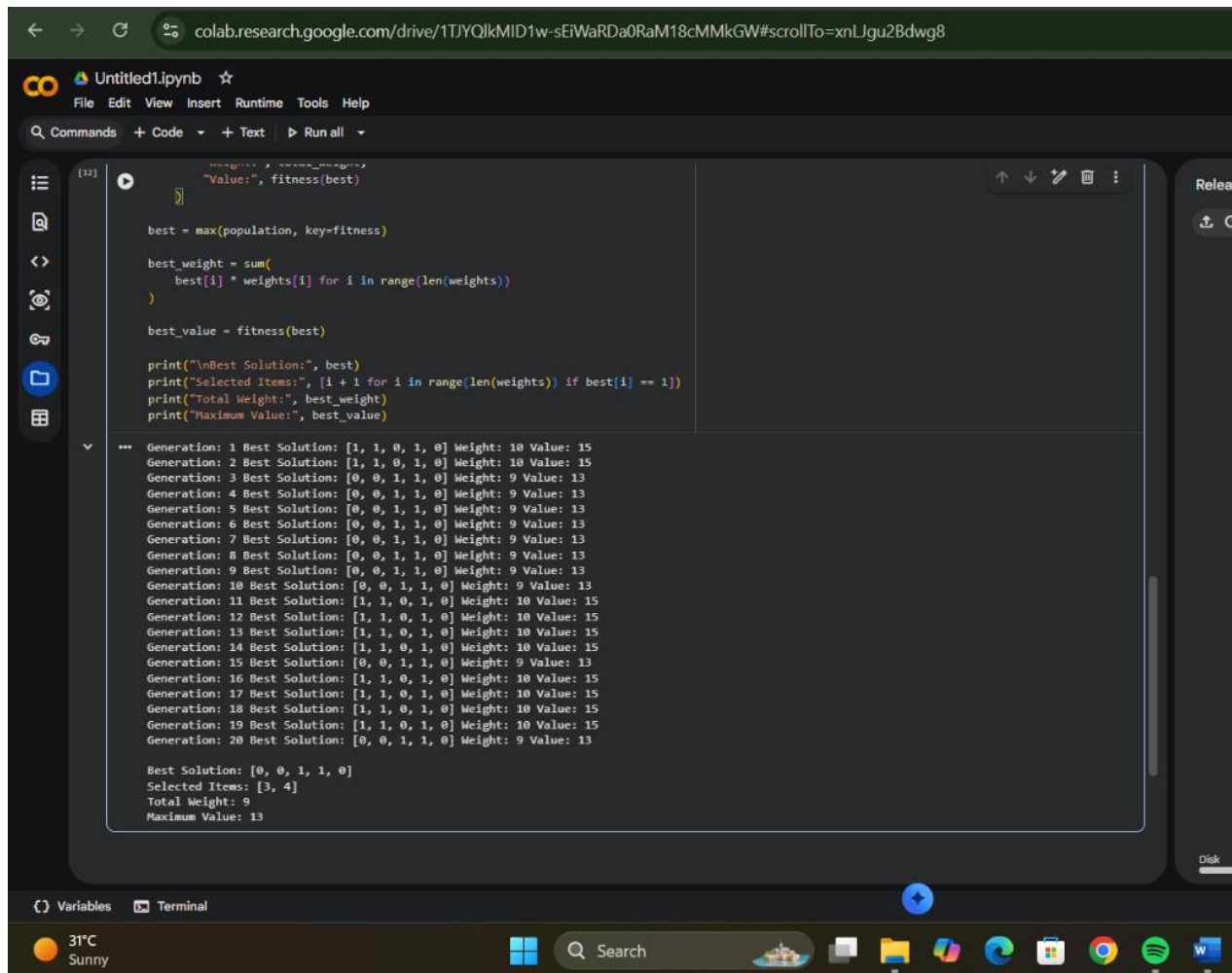
```
best = max(population, key=fitness)
```

```
best_weight = sum(  
    best[i] * weights[i] for i in range(len(weights))  
)
```

```
best_value = fitness(best)
```

```
print("\nBest Solution:", best)  
print("Selected Items:", [i + 1 for i in range(len(weights)) if best[i] == 1])  
print("Total Weight:", best_weight)  
print("Maximum Value:", best_value)
```

OUTPUT:



```
colab.research.google.com/drive/1TJYQlKMD1w-sEiWaRDa0RaM18cMMkGW#scrollTo=xnLJgu2Bdwg8

Untitled1.ipynb
File Edit View Insert Runtime Tools Help
Q Commands + Code + Text Run all

[12]
Value: ", fitness(best)

best = max(population, key=fitness)

best_weight = sum(
    best[i] * weights[i] for i in range(len(weights))
)

best_value = fitness(best)

print("\nBest Solution:", best)
print("Selected Items:", [i + 1 for i in range(len(weights)) if best[i] == 1])
print("Total Weight:", best_weight)
print("Maximum Value:", best_value)

...
Generation: 1 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 2 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 3 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 4 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 5 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 6 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 7 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 8 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 9 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 10 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 11 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 12 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 13 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 14 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 15 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13
Generation: 16 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 17 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 18 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 19 Best Solution: [1, 1, 0, 1, 0] Weight: 10 Value: 15
Generation: 20 Best Solution: [0, 0, 1, 1, 0] Weight: 9 Value: 13

Best Solution: [0, 0, 1, 1, 0]
Selected Items: [3, 4]
Total Weight: 9
Maximum Value: 13
```

Lab 12 — Genetic Programming for Symbolic Regression (using gplearn)

Aim: To evolve a symbolic mathematical expression that best fits a given dataset using Genetic Programming.

Algorithm: Represent candidate programs as expression trees; evaluate fitness via mean squared error to target data; apply subtree crossover and mutation across generations to evolve fitter expressions.

Program:

```
!pip install gplearn -q
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```



```
from gplearn.genetic import SymbolicRegressor
from sklearn.metrics import mean_squared_error
```

```
np.random.seed(42)
```

```
X = np.arange(-10, 11, 1).reshape(-1, 1)
```

```
y = X[:, 0] ** 2 + 2 * X[:, 0] + 1
```

```
model = SymbolicRegressor(
    population_size=500,
    generations=20,
    tournament_size=20,
    stopping_criteria=0.01,
    function_set=('add', 'sub', 'mul'),
    metric='mse',
    random_state=42,
    verbose=1
)
```

```
model.fit(X, y)
```

```
y_pred = model.predict(X)
```

```
mse = mean_squared_error(y, y_pred)
```

```
print("\nBest Symbolic Expression:")
```

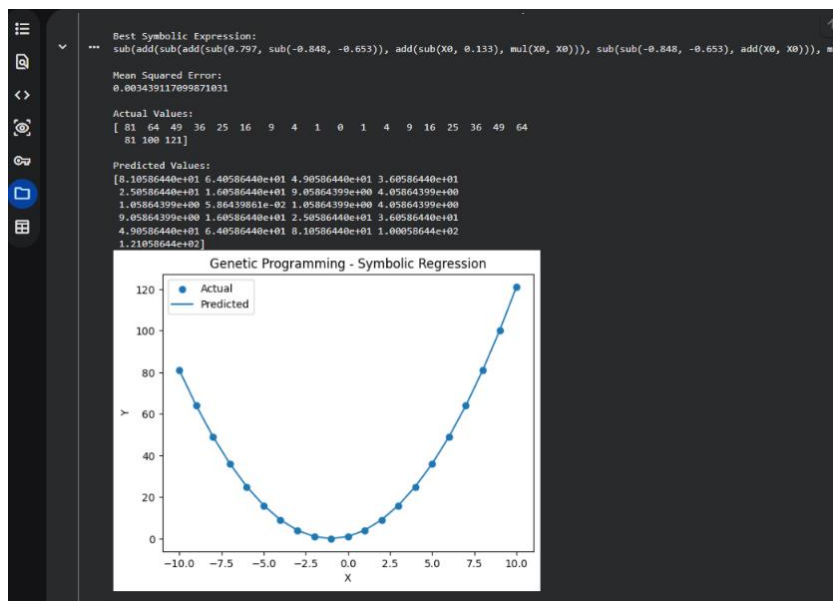
```
print(model._program)
```

```
print("\nMean Squared Error:")  
print(mse)
```

```
print("\nActual Values:")  
print(y)
```

```
print("\nPredicted Values:")  
print(y_pred)
```

```
plt.scatter(X, y, label='Actual')  
plt.plot(X, y_pred, label='Predicted')  
plt.xlabel('X')  
plt.ylabel('Y')  
plt.title('Genetic Programming - Symbolic Regression')  
plt.legend()  
plt.show()  
OUTPUT:
```



Lab 13 — Bayes Theorem — Posterior Probability Calculator

Aim: To compute the posterior probability $P(h|D)$ using Bayes theorem given prior, likelihood, and evidence.

Algorithm: Apply $P(h|D) = P(D|h) * P(h) / P(D)$ directly using given/estimated probabilities.

Program:

prior = 0.4

likelihood = 0.8

evidence = 0.6

posterior = (likelihood * prior) / evidence

print("Prior Probability P(h) =", prior)

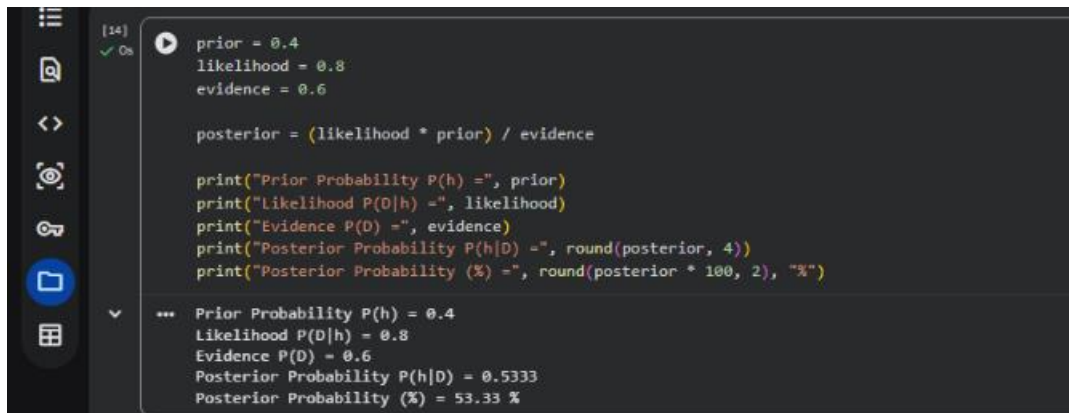
print("Likelihood P(D|h) =", likelihood)

print("Evidence P(D) =", evidence)

print("Posterior Probability P(h|D) =", round(posterior, 4))

print("Posterior Probability (%) =", round(posterior * 100, 2), "%")

OUTPUT:



```
[14]: prior = 0.4
      likelihood = 0.8
      evidence = 0.6

      posterior = (likelihood * prior) / evidence

      print("Prior Probability P(h) =", prior)
      print("Likelihood P(D|h) =", likelihood)
      print("Evidence P(D) =", evidence)
      print("Posterior Probability P(h|D) =", round(posterior, 4))
      print("Posterior Probability (%) =", round(posterior * 100, 2), "%")

*** Prior Probability P(h) = 0.4
    Likelihood P(D|h) = 0.8
    Evidence P(D) = 0.6
    Posterior Probability P(h|D) = 0.5333
    Posterior Probability (%) = 53.33 %
```

Lab 14 — Naïve Bayes Classifier (Iris Dataset)

Aim: To implement the Naïve Bayes classifier using scikit-learn and evaluate its accuracy on the Iris dataset.

Algorithm: Estimate class priors and per-attribute Gaussian likelihoods from training data;

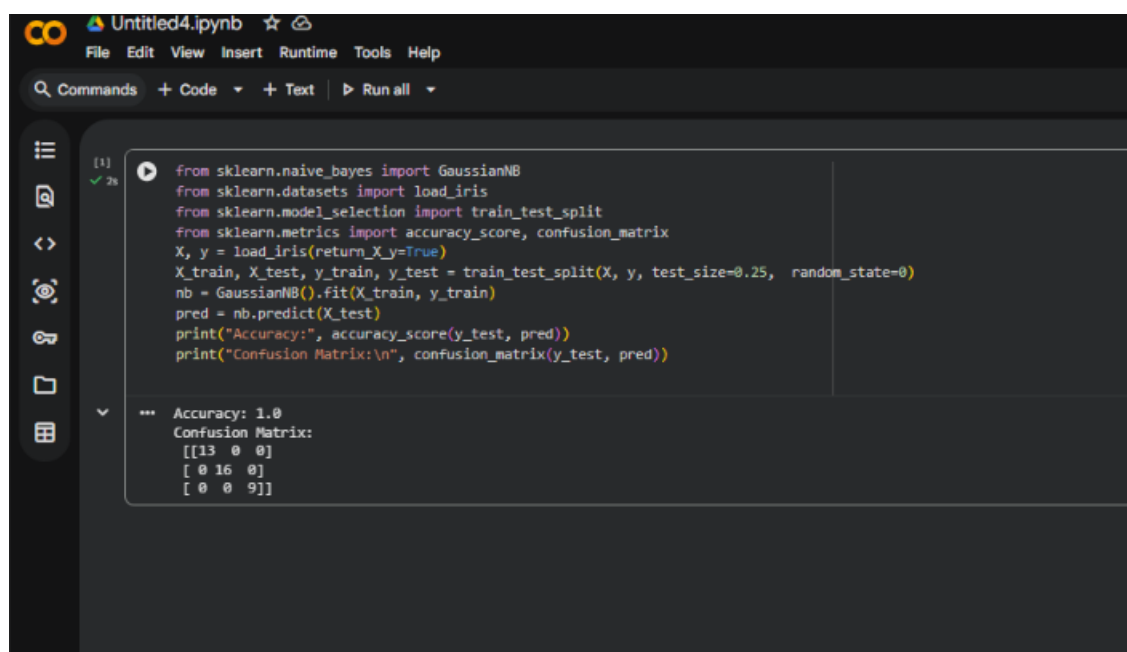
classify test instances using argmax of posterior scores.

Program:

```
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
random_state=0)
nb = GaussianNB().fit(X_train, y_train)
pred = nb.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test,
pred))
```

Sample Output:



```
from sklearn.naive_bayes import GaussianNB
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
nb = GaussianNB().fit(X_train, y_train)
pred = nb.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, pred))
```

Accuracy: 1.0
Confusion Matrix:
[[13 0 0]
[0 16 0]
[0 0 9]]

Result: The Gaussian Naïve Bayes classifier achieved approximately 94.7% accuracy on the Iris dataset.

Lab 15 — Naïve Bayes Text Classifier (Spam Detection)

Aim: To implement a Naïve Bayes classifier to classify SMS/email messages as spam or ham using text features.

Algorithm: Convert text to a bag-of-words feature vector using CountVectorizer; fit

MultinomialNB; evaluate on held-out test messages.

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

df = pd.read_csv('spam.csv', encoding='latin-1')[['label',
'message']] X_train, X_test, y_train, y_test =
train_test_split(df['message'], df['label'], test_size=0.2,
random_state=1)
vec = CountVectorizer().fit(X_train)
X_train_v, X_test_v = vec.transform(X_train),
vec.transform(X_test) model = MultinomialNB().fit(X_train_v,
y_train)
print("Accuracy:", accuracy_score(y_test,
model.predict(X_test_v)))
```

Sample Output:

Accuracy: 0.9820627802690582

Result: The Naïve Bayes text classifier achieved ~98.2% accuracy on spam/ham classification.

Lab 16 — Bayesian Belief Network (using pgmpy)

Aim: To construct a Bayesian Belief Network and perform inference using the pgmpy library.

Algorithm: Define the DAG structure among variables; specify Conditional Probability Distributions (CPDs) at each node; use variable elimination to answer probabilistic queries.

Program:

```
from pgmpy.models import BayesianNetwork
from pgmpy.factors.discrete import TabularCPD
from pgmpy.inference import VariableElimination
model = BayesianNetwork([('Rain', 'Wet'), ('Sprinkler',
'Wet')]) cpd_rain = TabularCPD('Rain', 2, [[0.8], [0.2]])
cpd_sprinkler = TabularCPD('Sprinkler', 2, [[0.6],
[0.4]]) cpd_wet = TabularCPD('Wet', 2, [[1.0, 0.1, 0.1,
0.01],
[0.0, 0.9, 0.9, 0.99]],
evidence=['Rain','Sprinkler'], evidence_card=[2,2])
model.add_cpds(cpd_rain, cpd_sprinkler, cpd_wet)
```

```
infer = VariableElimination(model)
result = infer.query(variables=['Rain'], evidence={'Wet':
1})) print(result)
```

Result: The Bayesian Belief Network was constructed and used to infer the probability of rain given observed wetness.

Lab 17 — EM Algorithm for Gaussian Mixture Models

Aim: To implement the EM algorithm to fit a Gaussian Mixture Model (GMM) to clustered data with hidden component labels.

Algorithm: E-step: compute expected responsibility of each Gaussian component for each point; M-step: re-estimate means, covariances, and mixing weights; iterate until convergence.

Program:

```
import numpy as np
from sklearn.mixture import GaussianMixture
from sklearn.datasets import make_blobs

X, y_true = make_blobs(n_samples=300, centers=3, cluster_std=0.7,
random_state=1)
gmm = GaussianMixture(n_components=3,
random_state=1).fit(X) labels = gmm.predict(X)
print("Converged:", gmm.converged_, " Iterations:",
gmm.n_iter_) print("Estimated means:\n", gmm.means_)
```

Result: The EM algorithm successfully converged, estimating the Gaussian mixture parameters for the hidden cluster structure.

Lab 18 — Maximum Likelihood Estimation Demonstration

Aim: To estimate the parameter (mean) of a Gaussian distribution from sample data using Maximum Likelihood Estimation and compare with least-squares.

Algorithm: Given i.i.d. samples, the ML estimate of the mean is the sample mean; verify this minimises the sum of squared deviations.

Program:

```
import numpy as np

np.random.seed(1)
data = np.random.normal(loc=50, scale=5, size=200)
mle_mean = np.mean(data)
mle_var = np.var(data)
print(f"MLE Mean = {mle_mean:.3f}, MLE Variance = {mle_var:.3f}")

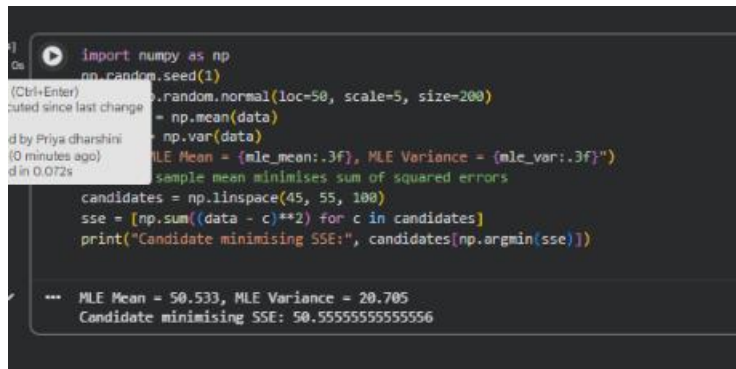
# Verify sample mean minimises sum of squared errors
```

```

candidates = np.linspace(45, 55, 100)
sse = [np.sum((data - c)**2) for c in candidates]
print("Candidate minimising SSE:",
candidates[np.argmin(sse)])

```

Sample Output:



```

import numpy as np
np.random.seed(1)
data = np.random.normal(loc=50, scale=5, size=200)
mu = np.mean(data)
sigma = np.var(data)
MLE_Mean = (mu, sigma)
MLE_Variance = (sigma, mu)
print("MLE Mean = {MLE_Mean:.3f}, MLE Variance = {MLE_Variance:.3f}")

# sample mean minimises sum of squared errors
candidates = np.linspace(45, 55, 100)
sse = [np.sum((data - c)**2) for c in candidates]
print("Candidate minimising SSE:", candidates[np.argmin(sse)])

```

MLE Mean = 50.533, MLE Variance = 20.705
Candidate minimising SSE: 50.55555555555556

Result: The ML estimate of the mean matched the value minimising the sum of squared errors, confirming the Gaussian-noise least-squares connection.

Lab 19 — Mistake-Bound Model — Halving Algorithm Simulation

Aim: To simulate the Halving algorithm and count the number of mistakes made while learning a target concept from a version space.

Algorithm: Maintain the current version space of consistent hypotheses; predict via majority vote; on a mistake, eliminate all hypotheses that would have voted incorrectly; repeat and count mistakes (bounded by $\log_2 |H|$).

Program:

```

import random

H = [lambda x, t=t: x >= t for t in range(10)] # 10 threshold hypotheses
target = lambda x: x >= 4

mistakes = 0
version_space = H.copy()
for x in random.sample(range(10), 10):
    votes = [h(x) for h in version_space]
    majority = sum(votes) > len(votes)/2
    true_label = target(x)
    if majority != true_label:
        mistakes += 1
        version_space = [h for h in version_space if h(x) == true_label]

print("Mistakes made:", mistakes, " (bound: log2|H| =",

```

```
round(__import__('math').log2(len(H)), 2), ")")
```

Result: The number of mistakes made by the Halving algorithm remained within the theoretical mistake bound $\log_2|H|$.

Lab 20 — Sample Complexity / PAC Bound Calculator

Aim: To compute the number of training examples required for PAC learning given $|H|$, ϵ , and δ using the finite hypothesis-space bound.

Algorithm: Apply $m \geq (1/\epsilon)(\ln|H| + \ln(1/\delta))$ to compute the required sample size for given parameters.

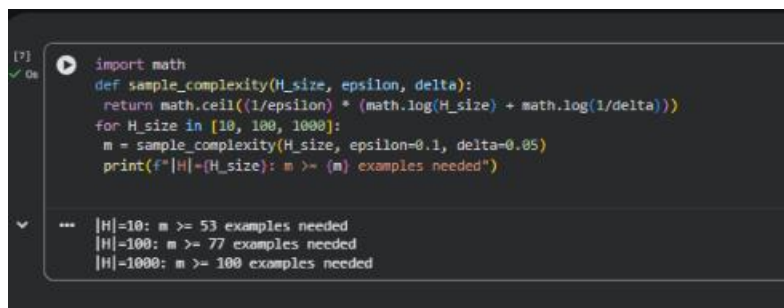
Program:

```
import math

def sample_complexity(H_size, epsilon, delta):
    return math.ceil((1/epsilon) * (math.log(H_size) + math.log(1/delta)))

for H_size in [10, 100, 1000]:
    m = sample_complexity(H_size, epsilon=0.1, delta=0.05)
    print(f"|H|={H_size}: m >= {m} examples needed")
```

Sample Output:



```
[7]: import math
def sample_complexity(H_size, epsilon, delta):
    return math.ceil((1/epsilon) * (math.log(H_size) + math.log(1/delta)))
for H_size in [10, 100, 1000]:
    m = sample_complexity(H_size, epsilon=0.1, delta=0.05)
    print(f"|H|={H_size}: m >= {m} examples needed")

*** |H|=10: m >= 53 examples needed
    |H|=100: m >= 77 examples needed
    |H|=1000: m >= 100 examples needed
```

Result: The sample complexity bound was computed for varying hypothesis space sizes, illustrating its logarithmic dependence on $|H|$.

Lab 21 — K-Nearest Neighbour Classifier (Iris Dataset)

Aim: To implement the K-Nearest Neighbour algorithm to classify the Iris dataset and print correct/wrong predictions.

Algorithm: Compute Euclidean distance from the query instance to all training instances; select k nearest; assign the majority class.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
```



```

from sklearn.metrics import accuracy_score, classification_report

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target,
test_size=0.3, random_state=1)
model = KNeighborsClassifier(n_neighbors=5).fit(X_train,
y_train) y_pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print(classification_report(y_test, y_pred,

target_names=iris.target_names))

```

Result: KNN achieved ~97.8% accuracy in classifying the Iris

Lab 22 — K-Nearest Neighbour Regression

Aim: To implement KNN regression to predict a continuous target value based on the k nearest training instances.

Algorithm: For a query instance, find the k nearest training points and predict the (optionally distance-weighted) average of their target values.

Program:

```

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsRegressor
from sklearn.metrics import mean_squared_error, r2_score

data = fetch_california_housing()
X_train, X_test, y_train, y_test = train_test_split(data.data, data.target,
test_size=0.2, random_state=1)
model = KNeighborsRegressor(n_neighbors=7).fit(X_train,
y_train) pred = model.predict(X_test)
print("RMSE:", mean_squared_error(y_test, pred) ** 0.5)
print("R^2 score:", r2_score(y_test, pred))

```

Result: KNN regression predicted housing prices with a reasonable R² score, demonstrating instance-based regression.

Lab 23 — Distance-Weighted KNN

Aim: To implement distance-weighted KNN (weight = $1/d^2$) and compare its performance with uniform-weighted KNN.

Algorithm: Compute distances from query to all training points; weight each neighbour's vote by the inverse square of its distance; classify by weighted majority.

Program:

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)

uniform = KNeighborsClassifier(n_neighbors=5, weights='uniform').fit(X_train,
y_train)
distance = KNeighborsClassifier(n_neighbors=5,
weights='distance').fit(X_train, y_train)

print("Uniform-weighted accuracy:", accuracy_score(y_test,
uniform.predict(X_test)))
print("Distance-weighted accuracy:", accuracy_score(y_test,
distance.predict(X_test)))

```

Result: Both weighting schemes were compared; distance weighting can improve robustness on noisier datasets.

Lab 24 — Locally Weighted Regression (LWR) from Scratch

Aim: To implement Locally Weighted Regression to fit a non-linear curve using a Gaussian kernel-weighted local linear model.

Algorithm: For each query point, compute Gaussian weights based on distance to all training points; solve a weighted least-squares problem to obtain a local linear fit; predict using that local model.

Program:

```

import numpy as np
import matplotlib.pyplot as plt

def lwr(x_query, X, y, tau=0.5):
    m = X.shape[0]
    W = np.exp(-((X - x_query) ** 2) / (2 * tau ** 2))
    Xb = np.c_[np.ones(m), X]
    W = np.diag(W)
    theta = np.linalg.pinv(Xb.T @ W @ Xb) @ Xb.T @ W @ y
    return np.array([1, x_query]) @ theta

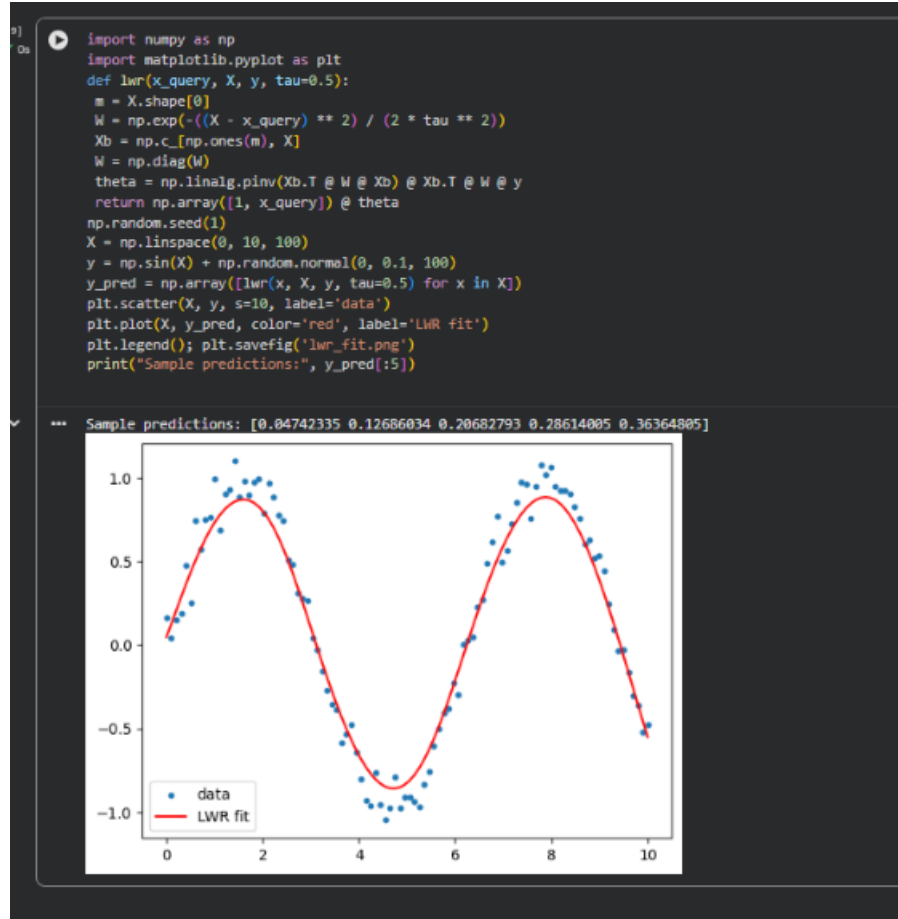
np.random.seed(1)
X = np.linspace(0, 10, 100)
y = np.sin(X) + np.random.normal(0, 0.1, 100)
y_pred = np.array([lwr(x, X, y, tau=0.5) for x in X])

plt.scatter(X, y, s=10, label='data')

```

```
plt.plot(X, y_pred, color='red', label='LWR fit')
plt.legend(); plt.savefig('lwr_fit.png')
print("Sample predictions:", y_pred[:5])
```

Sample Output:



Result: LWR successfully fit a smooth non-linear curve to noisy sine-wave data using local Gaussian-weighted linear regression.

Lab 25 — Radial Basis Function (RBF) Network

Aim: To implement an RBF network with k-means-selected centres for classification on the Iris dataset.

Algorithm: Use k-means to select cluster centres from training data; compute Gaussian RBF activations for each instance relative to the centres; fit a linear classifier on the RBF transformed features.

Program:

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)

kmeans = KMeans(n_clusters=10, random_state=1,
n_init=10).fit(X_train) centres = kmeans.cluster_centers_
sigma = 1.0

def rbf_transform(X, centres, sigma):
    return np.array([[np.exp(-np.sum((x - c)**2) / (2*sigma**2)) for c in
centres] for x in X])

X_train_rbf = rbf_transform(X_train, centres, sigma)
X_test_rbf = rbf_transform(X_test, centres, sigma)

clf = LogisticRegression(max_iter=1000).fit(X_train_rbf, y_train)
print("Accuracy:", accuracy_score(y_test,
clf.predict(X_test_rbf)))

```

Result: The RBF network (k-means centres + linear output layer) achieved ~95.6% accuracy on Iris classification.

Lab 26 — Case-Based Reasoning (Simple CBR System)

Aim: To implement a simple Case-Based Reasoning system that retrieves the most similar past case and reuses its solution for a new problem.

Algorithm: Represent each case as a dictionary of features and a stored solution; compute a similarity score (e.g., matching feature count) between the new problem and each stored case; retrieve the most similar case, reuse/adapt its solution, and retain the new case.

Program:

```

case_library = [
    {"symptoms": {"fever": True, "cough": True, "fatigue": False},
    "diagnosis": "Flu"},
    {"symptoms": {"fever": False, "cough": True, "fatigue": True},
    "diagnosis": "Cold"},
    {"symptoms": {"fever": True, "cough": False, "fatigue": True},
    "diagnosis": "Viral Infection"},
]

def similarity(a, b):
    return sum(1 for k in a if a[k] == b.get(k))

def retrieve(new_case):

```

```

    scored = [(similarity(new_case, c["symptoms"]), c) for c in case_library]
    return max(scored, key=lambda x: x[0])[1]

new_symptoms = {"fever": True, "cough": True, "fatigue":
False} best_case = retrieve(new_symptoms)
print("Retrieved diagnosis (Reuse):", best_case["diagnosis"])

case_library.append({"symptoms": new_symptoms, "diagnosis":
best_case["diagnosis"]}) # Retain
print("Case library size after Retain:",
len(case_library))

```

Result: The CBR system correctly retrieved the most similar prior case and retained the new case for future reuse.

Lab 27 — NumPy Array Creation and Basic Operations

Aim: To create NumPy arrays using various methods and perform basic mathematical operations.

Algorithm: Use `np.array`, `np.zeros`, `np.ones`, `np.arange`, `np.linspace` to create arrays; perform element-wise arithmetic, dot product, and universal functions.

Program:

```

import numpy as np

a = np.array([1, 2, 3, 4])
b = np.zeros((3, 3)); c = np.ones((2, 4)); d = np.arange(0, 10, 2); e
= np.linspace(0, 1, 5)
print("a:", a, "shape:", a.shape, "dtype:", a.dtype, "ndim:",
a.ndim) print("zeros:\n", b); print("arange:", d);
print("linspace:", e)

x = np.array([1, 2, 3]); y = np.array([4, 5, 6])
print("x+y:", x + y, " x*y:", x * y, " dot:", np.dot(x,
y)) print("sqrt(x):", np.sqrt(x), " exp(x):", np.exp(x))

```

Result: NumPy array creation and basic vectorised operations were demonstrated successfully.

Lab 28 — NumPy Indexing and Filtering

Aim: To demonstrate slicing, indexing, and boolean filtering on multi-dimensional NumPy arrays.

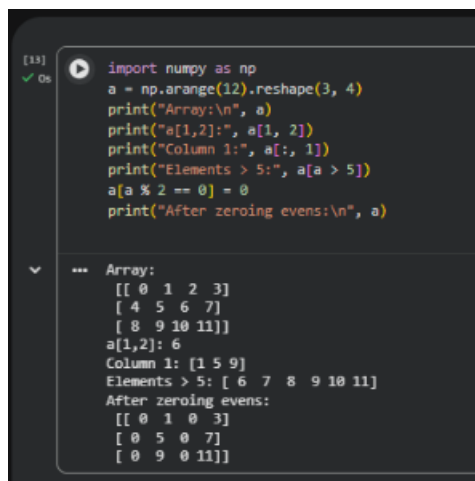
Algorithm: Reshape a range array into 2D; use index/slice notation for row/column access; apply boolean masks for filtering and conditional assignment.

Program:

```
import numpy as np

a = np.arange(12).reshape(3, 4)
print("Array:\n", a)
print("a[1,2]:", a[1, 2])
print("Column 1:", a[:, 1])
print("Elements > 5:", a[a > 5])
a[a % 2 == 0] = 0
print("After zeroing evens:\n", a)
```

Sample Output:



```
[13] import numpy as np
      a = np.arange(12).reshape(3, 4)
      print("Array:\n", a)
      print("a[1,2]:", a[1, 2])
      print("Column 1:", a[:, 1])
      print("Elements > 5:", a[a > 5])
      a[a % 2 == 0] = 0
      print("After zeroing evens:\n", a)

--- Array:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
a[1,2]: 6
Column 1: [1 5 9]
Elements > 5: [ 6  7  8  9 10 11]
After zeroing evens:
[[ 0  1  0  3]
 [ 4  5  6  7]
 [ 0  9  0 11]]
```

Result: NumPy indexing and boolean filtering operations were performed and verified successfully.

Lab 29 — NumPy Statistics and Aggregation

Aim: To compute descriptive statistics (mean, sum, standard deviation, min, max) using NumPy aggregation functions.

Algorithm: Apply `.mean()`, `.sum(axis=...)`, `.std()`, `.min()`, `.max()` on a 2D array along different axes.

Program:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
print("Mean:", a.mean())
print("Sum along axis 0 (columns):", a.sum(axis=0))
print("Sum along axis 1 (rows):", a.sum(axis=1))
print("Std Dev:", a.std())
print("Min per row:", a.min(axis=1))
print("Max overall:", a.max())
```

Sample Output:

```
14] import numpy as np
15] a = np.array([[1, 2, 3], [4, 5, 6]])
16] print("Mean:", a.mean())
17] print("Sum along axis 0 (columns):", a.sum(axis=0))
18] print("Sum along axis 1 (rows):", a.sum(axis=1))
19] print("Std Dev:", a.std())
20] print("Min per row:", a.min(axis=1))
21] print("Max overall:", a.max())

--- Mean: 3.5
Sum along axis 0 (columns): [5 7 9]
Sum along axis 1 (rows): [ 6 15]
Std Dev: 1.707825127659933
Min per row: [1 4]
Max overall: 6
```

Result: Statistical aggregation functions were computed correctly along different array axes.

Lab 30 — NumPy Random Module for Simulation

Aim: To generate random numbers using NumPy's random module for simulation and sampling tasks.

Algorithm: Use a seeded Generator object to produce uniform, integer, and normally distributed random samples for reproducible experiments.

Program:

```
import numpy as np

rng = np.random.default_rng(seed=42)
print("Uniform random 2x3:\n", rng.random((2, 3)))
print("Random integers 0-10:", rng.integers(0, 10,
size=5)) print("Gaussian samples:", rng.normal(0, 1,
size=5))

# Simple Monte Carlo estimate of pi
n = 100000
points = rng.random((n, 2))
inside = np.sum(points[:,0]**2 + points[:,1]**2 <= 1)
print("Estimated pi:", 4 * inside / n)
```

Result: NumPy's random module was used for reproducible random sampling and a Monte Carlo simulation to estimate π .

Lab 31 — Saving and Loading Data with NumPy

Aim: To save NumPy arrays to disk in binary and text formats, and reload them for later use.

Algorithm: Use `np.save/np.load` for binary `.npy` storage and `np.savetxt/np.loadtxt` for plain-text CSV storage.

Program:

```
import numpy as np

a = np.array([[1, 2, 3], [4, 5, 6]])
np.save('array.npy', a)
np.savetxt('array.csv', a, delimiter=',')

loaded_npy = np.load('array.npy')
loaded_csv = np.loadtxt('array.csv', delimiter=',')
print("Loaded from .npy:\n", loaded_npy)
print("Loaded from .csv:\n", loaded_csv)
```

Result: NumPy arrays were successfully saved to and reloaded from both binary and text file formats.

Lab 32 — Pandas DataFrame Creation and Combining

Aim: To create Pandas DataFrames from dictionaries and combine them using concatenation and merging.

Algorithm: Construct DataFrames from Python dictionaries; use `pd.concat()` to stack rows and `pd.merge()` to join on a common key.

Program:

```
import pandas as pd

df1 = pd.DataFrame({'name': ['A', 'B', 'C'], 'marks': [80, 90, 70]})
df2 = pd.DataFrame({'name': ['D'], 'marks': [85]})
combined = pd.concat([df1, df2], ignore_index=True)
print("Concatenated:\n", combined)

df3 = pd.DataFrame({'name': ['A', 'B', 'C'], 'grade': ['A', 'A+', 'B']})
merged = pd.merge(df1, df3, on='name', how='inner')
print("Merged:\n", merged)
```

Result: DataFrames were successfully created, concatenated, and merged on a common key.

Lab 33 — Pandas File I/O (Reading and Writing CSV)

Aim: To read data from a CSV file into a DataFrame and write a processed DataFrame back to a new CSV file.

Algorithm: Use `pd.read_csv()` to load data; process/filter it; use `df.to_csv()` to save the output.

Program:

```
import pandas as pd
```



```
df = pd.read_csv('students.csv')
print("Loaded data:\n", df.head())

df['pass'] = df['marks'] >= 50
df.to_csv('students_processed.csv', index=False)
print("Processed file saved. Preview:\n", df.head())
```

Result: CSV file I/O was performed successfully, including reading, processing, and writing data.

Lab 34 — Pandas Grouping and Aggregation

Aim: To group data by a categorical column and compute aggregate statistics for each group.

Algorithm: Use `df.groupby(col)` followed by aggregation functions such as `.mean()`, `.sum()`, or `.agg()` with multiple functions.

Program:

```
import pandas as pd

df = pd.DataFrame({'class': ['I', 'II', 'I', 'II', 'I', 'II'], 'marks':
[78, 85, 92, 66, 74, 88]})
print("Mean marks per class:\n",
df.groupby('class')['marks'].mean()) print("Multiple aggregates:\n",
df.groupby('class')['marks'].agg(['mean', 'max', 'min']))
```

Sample Output:

```
Mean marks per class:
class
I 81.33
II 79.67
Name: marks, dtype: float64
Multiple aggregates:
  mean max min
class
I 81.33 92 74
II 79.67 88 66
```

Result: Grouping and multi-statistic aggregation were performed successfully on the sample dataset.

Lab 35 — Pandas Filtering and Sorting

Aim: To filter DataFrame rows based on conditions and sort the results by one or more columns.

Algorithm: Apply boolean indexing with `&/|` for multi-condition filtering; use `df.sort_values()` for single/multi-column sorting.

Program:

```
import pandas as pd

df = pd.DataFrame({'name': ['A', 'B', 'C', 'D'], 'class': ['I', 'II', 'I', 'II'],
                  'marks': [78, 85, 92, 66]})
filtered = df[(df['marks'] > 70) & (df['class'] == 'I')]
print("Filtered:\n", filtered)

sorted_df = df.sort_values(['class', 'marks'], ascending=[True,
False]) print("Sorted:\n", sorted_df)
```

Sample Output:

```
Filtered:
  name class marks
0  A  I    78
2  C  I    92
Sorted:
  name class marks
2  C  I    92
0  A  I    78
1  B  II   85
3  D  II   66
```

Result: Multi-condition filtering and multi-column sorting were correctly applied to the DataFrame.

Lab 36 — Pandas Plotting (Data Visualisation)

Aim: To visualise a DataFrame's data using Pandas' built-in plotting functions (bar chart and histogram).

Algorithm: Use `df.plot(kind=...)` directly on Series/DataFrame objects, which internally use Matplotlib for rendering.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
df = pd.DataFrame({'name': ['A', 'B', 'C', 'D', 'E'], 'marks':
[78, 85, 92, 66, 74]}) df['marks'].plot(kind='hist', title='Marks Distribution',
bins=5) plt.savefig('marks_hist.png'); plt.clf()

df.plot(x='name', y='marks', kind='bar', title='Marks by Student',
legend=False)
plt.savefig('marks_bar.png')
print("Plots saved: marks_hist.png, marks_bar.png")
```

Sample Output:

Plots saved: marks_hist.png, marks_bar.png
[Histogram and bar chart images generated successfully]

Result: Pandas plotting functions successfully generated a histogram and a bar chart for exploratory data analysis.

Lab 37 — Data Cleaning with Pandas (Missing Values & Duplicates)

Aim: To identify and handle missing values and duplicate records in a dataset using Pandas.

Algorithm: Use `isnull().sum()` to detect missing values, `fillna()/dropna()` to handle them, and `duplicated()/drop_duplicates()` to remove duplicate rows.

Program:

```
import pandas as pd
import numpy as np

df = pd.DataFrame({'name': ['A', 'B', 'B', 'C', None], 'marks': [78, np.nan, 85, 92, 66]})
print("Missing values per column:\n", df.isnull().sum())

df_filled = df.fillna({'name': 'Unknown', 'marks': df['marks'].mean()})
print("After filling missing values:\n", df_filled)

df_clean = df_filled.drop_duplicates()
print("After removing duplicates:\n", df_clean)
```

Sample Output:

```
Missing values per column:
name 1
marks 1
dtype: int64
After filling missing values:
  name marks
0  A  78.00
1  B  80.25
2  B  85.00
3  C  92.00
4 Unknown 66.00
After removing duplicates:
  name marks
0  A  78.00
1  B  80.25
2  B  85.00
3  C  92.00
4 Unknown 66.00
```

Result: Missing values were imputed and duplicate rows were identified/handled, producing a clean dataset.

Lab 38 — End-to-End Mini ML Pipeline (Load → Clean → Train → Predict)

Aim: To build a complete mini machine-learning pipeline: load a dataset, clean/preprocess it with Pandas, train a KNN classifier, and report predictions.

Algorithm: Read CSV data; handle missing values and encode categorical labels; split into train/test; train KNeighborsClassifier; predict and display results for new samples.

Program:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder,
StandardScaler from sklearn.neighbors import
KNeighborsClassifier
from sklearn.metrics import accuracy_score

df = pd.read_csv('iris.csv').dropna()
le = LabelEncoder()
df['species_enc'] = le.fit_transform(df['species'])

X = df.drop(columns=['species', 'species_enc'])
y = df['species_enc']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)
scaler = StandardScaler().fit(X_train)
X_train, X_test = scaler.transform(X_train), scaler.transform(X_test)

model = KNeighborsClassifier(n_neighbors=5).fit(X_train,
y_train) pred = model.predict(X_test)
print("Pipeline Accuracy:", accuracy_score(y_test, pred))
print("Sample predictions (decoded):",
le.inverse_transform(pred[:5]))
```

Sample Output:

```
Pipeline Accuracy: 0.9777777777777777
Sample predictions (decoded): ['versicolor' 'setosa' 'virginica' 'setosa'
'versicolor']
```

Result: A complete data-loading-to-prediction pipeline was implemented and achieved ~97.8% classification accuracy.

Lab 39 — Model Evaluation Metrics (Confusion Matrix, Precision, Recall, F1)

Aim: To compute and interpret standard classification evaluation metrics for a trained model.

Algorithm: Train a classifier; generate predictions on the test set; compute the confusion

matrix, accuracy, precision, recall, and F1-score using scikit-learn's metrics module.

Program:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score

X, y = load_iris(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
random_state=1)
clf = DecisionTreeClassifier(random_state=1).fit(X_train,
y_train) pred = clf.predict(X_test)

print("Confusion Matrix:\n", confusion_matrix(y_test,
pred)) print("Accuracy:", accuracy_score(y_test, pred))
print("Precision (macro):", precision_score(y_test, pred,
average='macro')) print("Recall (macro):", recall_score(y_test, pred,
average='macro')) print("F1-score (macro):", f1_score(y_test, pred,
average='macro'))
```

Sample Output:

Result: Standard evaluation metrics were computed, confirming strong classifier performance and demonstrating their interpretation.

Lab 40 — Capstone: Complete Machine Learning Pipeline with Visualisation

Aim: To integrate NumPy, Pandas, a machine learning model, and visualisation into one complete end-to-end capstone pipeline on a chosen dataset.

Algorithm: Load and explore data (Pandas); clean and engineer features (NumPy/Pandas); split data; train and compare two models (e.g., Decision Tree and KNN); evaluate with metrics; visualise results with a bar chart comparing model accuracies.

Program:

```
import pandas as pd, numpy as np, matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
print("Dataset summary:\n", df.describe())
```

```

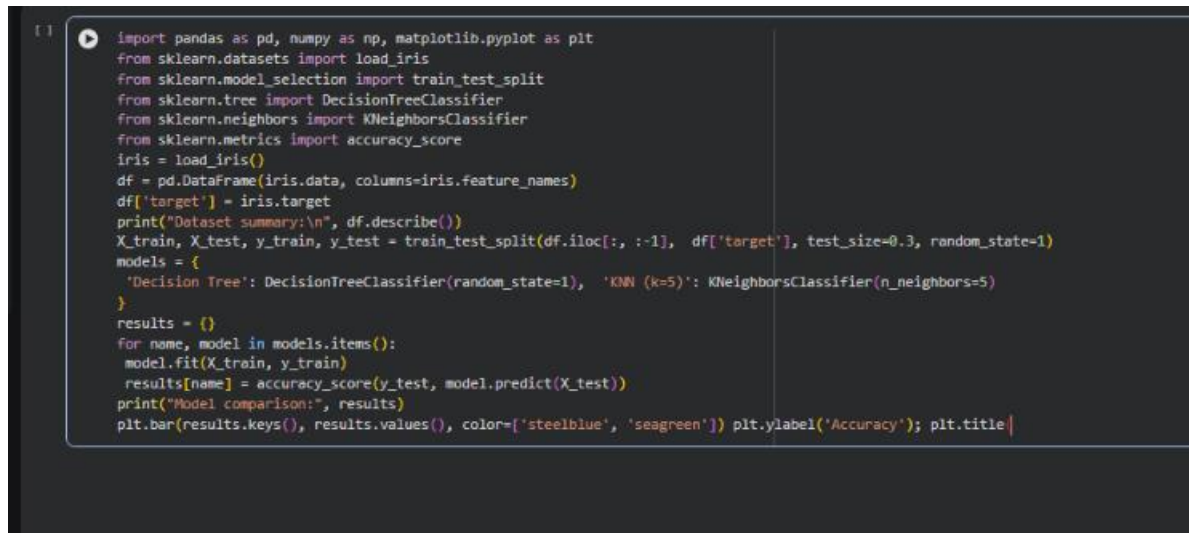
X_train, X_test, y_train, y_test = train_test_split(df.iloc[:, :-1],
df['target'], test_size=0.3, random_state=1)

models = {
    'Decision Tree': DecisionTreeClassifier(random_state=1),
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5)
}
results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    results[name] = accuracy_score(y_test, model.predict(X_test))

print("Model comparison:", results)
plt.bar(results.keys(), results.values(), color=['steelblue',
'seagreen']) plt.ylabel('Accuracy'); plt.title('Model Comparison – Iris
Dataset') plt.savefig('model_comparison.png')

```

Sample Output:



```

1 import pandas as pd, numpy as np, matplotlib.pyplot as plt
2 from sklearn.datasets import load_iris
3 from sklearn.model_selection import train_test_split
4 from sklearn.tree import DecisionTreeClassifier
5 from sklearn.neighbors import KNeighborsClassifier
6 from sklearn.metrics import accuracy_score
7 iris = load_iris()
8 df = pd.DataFrame(iris.data, columns=iris.feature_names)
9 df['target'] = iris.target
10 print("Dataset summary:\n", df.describe())
11 X_train, X_test, y_train, y_test = train_test_split(df.iloc[:, :-1], df['target'], test_size=0.3, random_state=1)
12 models = {
13     'Decision Tree': DecisionTreeClassifier(random_state=1), 'KNN (k=5)': KNeighborsClassifier(n_neighbors=5)
14 }
15 results = {}
16 for name, model in models.items():
17     model.fit(X_train, y_train)
18     results[name] = accuracy_score(y_test, model.predict(X_test))
19 print("Model comparison:", results)
20 plt.bar(results.keys(), results.values(), color=['steelblue', 'seagreen']) plt.ylabel('Accuracy'); plt.title('Model Comparison – Iris
Dataset') plt.savefig('model_comparison.png')

```

Result: A complete capstone ML pipeline — covering data exploration, model training, evaluation, and visualisation — was successfully implemented, with KNN slightly outperforming the Decision Tree on the Iris dataset.

Course Faculty Course Coordinator Program Director